KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

ASSIGNMENT II – CASE STUDY

Student Academic Performance Analysis and Dropout Prediction Using Exploratory Data Analysis and Deep Learning

Case Study: Exploratory Data Analysis and Visualization of a Real-World Dataset

Student Name	RITHI R
Roll Number / College ID	24AD091
Department	Artificial Intelligence and Data Science
Year / Semester	III Year / Semester V (Batch 2023–2027)
Course Code	U21ADP05
Course Title	Exploratory Data Analysis and Visualization
Assignment	Assignment II – Case Study (40 marks)
Academic Year	2026–2027 (Odd Semester)
Course Coordinator	Mr. Rushikesh Kadam
Report Date	18 October 2026

Abstract

Student dropout is a costly problem for higher-education institutions, and early identification of at-risk students allows timely support. This study analyses the UCI dataset "Predict Students' Dropout and Academic Success" (4,424 students, 36 features, three outcomes: Dropout, Enrolled, Graduate). Exploratory analysis found no missing values or duplicates, then examined outliers, distributions, correlations and associations using seven-stage visualisation, statistical tests and PCA. Tuition-fee status, scholarship status and curricular units approved were among the strongest correlates of outcome. A multilayer perceptron tuned over 12 configurations achieved 75.6% test accuracy, macro-F1 0.715 and macro ROC-AUC 0.895, similar to logistic-regression and random-forest baselines. Results are from one stratified split.

Keywords: exploratory data analysis, data visualization, student dropout, multilayer perceptron, statistical analysis.

(Abstract: 105 words)

Table of Contents

1. Introduction and Objectives....7
1.1 Background....7
1.2 Problem Statement....7
1.3 Motivation....7
1.4 Objectives....7
1.5 Scope and Practical Relevance....7
2. Dataset Description....8
2.1 Dataset Name, Source and Licence....8
2.2 Why This Dataset Was Chosen....8
2.3 Verified Structure of the Dataset....8
2.4 Description of the Fields....9
2.5 Basic Descriptive Statistics....10
3. Exploratory Data Analysis and Preprocessing....11
3.1 Loading and Inspecting the Data....11
3.2 Missing Values and Duplicates....11
3.3 Outlier Identification and Treatment....11
3.4 Cleaning, Transformation and Encoding....12
3.5 Train / Validation / Test Split and Leakage Prevention....12
3.6 Key Findings from EDA....12
4. Data Visualization....13
4.1 The Seven Stages of Visualization....13
4.2 Visualization Design Choices (Refine stage)....13
4.3 Interactive Visualizations....13
4.4 Visualization Gallery with Explanations....14
5. Statistical and Distribution-Based Analysis....24
5.1 Univariate Analysis....24
5.2 Probability Distributions....25
5.3 Bivariate Analysis....25
5.4 Multivariate Analysis....26
5.5 Summary of Statistical Findings....26
6. Deep Learning Model....26
6.1 Why a Multilayer Perceptron?....26
6.2 Inputs, Target and Pipeline....26
6.3 Architecture and Hyperparameters....27
6.4 Training, Validation and Testing Procedure....27
6.5 Basic Hyperparameter Tuning....28
6.6 Feature Analysis and Feature Importance....28
7. Model Evaluation and Result Visualization....29
7.1 Evaluation Metrics....29
7.2 Comparison with Baseline Models....29
7.3 Result Visualizations and Interpretation....30
8. Conclusion, Challenges and Future Scope....36
8.1 What Was Learned from the Data....36
8.2 Model Performance....36
8.3 Challenges and Limitations....36
8.4 Future Scope....36
9. Literature Review and References....37

9.1 Short Literature Review 37
9.2 References (APA 7th edition) 37
10. Rubric Alignment 37
Appendix A – Complete Python Source Code 39
A.1 config.py – paths, seed, column groups, plot style 39
A.2 data_preprocessing.py – loading, cleaning, split, pipeline 39
A.3 eda.py – statistics and distribution fitting 40
A.4 visualizations.py – Figures 1–12 and interactive pages 42
A.5 train_model.py – tuning, training, baselines 44
A.6 evaluate_model.py – evaluation, Figures 13–19 45
A.7 diagrams.py – workflow and architecture diagrams 46
A.8 run_all.py – run the whole pipeline 47
Appendix B – GitHub Repository and Checklist 48
B.1 Repository Structure 48
B.2 README Outline 48
B.3 Commands to Publish 48
B.4 GitHub Submission Checklist 49
Appendix C – Final Checklist Against the Assignment Requirements 49

List of Figures

Diagram 1: End-to-end project workflow 11
Figure 1: Student outcome distribution 14
Figure 2: Histograms of key numeric variables 15
Figure 3: Boxplots of key variables by outcome 16
Figure 4: Share of IQR outliers per numeric feature 16
Figure 5: Continuous distribution: Admission grade 17
Figure 6: Discrete distributions: Poisson and Binomial 18
Figure 7: Spearman correlation heatmap 19
Figure 8: Outcome mix by binary factors 20
Figure 9: First vs second semester grade 21
Figure 10: PCA projection of numeric features 22
Figure 11: Pairplot of key variables 23
Figure 12: Dropout rate by course code 24
Diagram 2: MLP architecture of the final model 27
Figure 13: Training vs validation loss 30
Figure 14: Training vs validation accuracy 31
Figure 15: Confusion matrix on the test set 32
Figure 16: One-vs-rest ROC curves 33
Figure 17: Prediction confidence: correct vs wrong 34
Figure 18: Permutation importance (top 12 features) 35
Figure 19: Hyperparameter search results 35

List of Tables

Table 1: Dataset identification 8
Table 2: Verified facts about the loaded file 8
Table 3: All 36 input fields and the target 9
Table 4: Descriptive statistics of numeric features 10
Table 5: Outlier summary (eight features with the highest IQR outlier share) 11
Table 6: Preprocessing applied to each group of features 12
Table 7: Data split (stratified, random_state = 42) 12
Table 8: Seven stages of visualization applied to the student dataset 13
Table 9: Distribution analysis results 25
Table 10: Twelve features most strongly associated with the outcome 25
Table 11: Statistical findings in plain words 26
Table 12: Model inputs and output 26
Table 13: Final model configuration 27
Table 14: Hyperparameter search results (sorted by validation macro-F1) 28
Table 15: Top 10 features by permutation importance (test set) 28
Table 16: MLP performance on validation and test sets 29
Table 17: Per-class results on the test set 29
Table 18: Test-set comparison of the MLP and baseline models (same split and preprocessing) 29
Table 19: Rubric alignment 38
Table 20: Purpose of each folder and file 48
Table 21: Checklist before submitting the link 49
Table 22: Requirement checklist 49

List of Abbreviations

Abbreviation	Meaning
ANOVA	Analysis of variance
AUC	Area under the ROC curve
CO	Course outcome
CSV	Comma-separated values
EDA	Exploratory data analysis
GDP	Gross domestic product
IQR	Inter-quartile range
MLP	Multilayer perceptron
PCA	Principal component analysis
ReLU	Rectified linear unit
ROC	Receiver operating characteristic
UCI	University of California, Irvine (Machine Learning Repository)
VIF	Variance inflation factor
CU	Curricular units (used in chart labels)

1. Introduction and Objectives

1.1 Background

Universities and colleges collect large amounts of data about their students: how they applied, their family background, their marks in each semester and even the economic conditions in the year they enrolled. When this data is explored carefully, it can reveal why some students graduate on time while others leave without a degree. Exploratory Data Analysis (EDA), introduced by John Tukey (1977), is the practice of looking at data through summaries and pictures before building any model, so that patterns, errors and surprises are found early.

1.2 Problem Statement

Given demographic, socio-economic, macro-economic and first-year academic information about a student, (a) which factors are most strongly associated with dropping out, and (b) can a deep-learning model predict whether the student will finally be a **Dropout**, still **Enrolled**, or a **Graduate** at the end of the normal course duration?

1.3 Motivation

- Dropout wastes the student's time and money and reduces an institution's graduation rate.
- If risky patterns are visible early (for example unpaid tuition fees or no approved course units in the first semester), tutors can offer help before it is too late.
- The dataset is real, contains a mix of numerical and categorical fields, and is large enough for both statistical analysis and a neural network, which makes it a good fit for this course.

1.4 Objectives

1. Select and verify a real-world dataset with at least 15 features (this one has 36).
2. Perform data understanding, cleaning, outlier analysis, encoding, scaling and a train/validation/test split without data leakage.
3. Create more than five meaningful visualisations (this report has 12 EDA figures and 7 model figures) that follow all seven stages of visualisation.
4. Carry out univariate, bivariate and multivariate statistical analysis and study discrete and continuous probability distributions.
5. Build a multilayer perceptron (deep-learning model), tune it, analyse feature importance and evaluate it with loss/accuracy curves, a confusion matrix and ROC/AUC.
6. Publish the work in a well-structured GitHub repository with a README.

1.5 Scope and Practical Relevance

The study covers one dataset from one Portuguese higher-education institution and focuses on understanding and predicting the final outcome. It is a learning project, not a deployed system; however, the same workflow (clean → explore → visualise → model → evaluate) is what an institutional analytics team would use to build an early-warning dashboard for tutors.

2. Dataset Description

2.1 Dataset Name, Source and Licence

Table 1: Dataset identification

Item	Details
Name	Predict Students' Dropout and Academic Success
Repository	UCI Machine Learning Repository (dataset id 697)
URL	https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success
DOI	10.24432/C5MC89
Creators	Valentim Realinho, Mónica Vieira Martins, Jorge Machado, Luís Baptista (Polytechnic Institute of Portalegre, Portugal)
Licence	Creative Commons Attribution 4.0 (CC BY 4.0)
Companion paper	Realinho, Machado, Baptista & Martins (2022), Data 7(11), 146, https://doi.org/10.3390/data7110146
Local file	data/data.csv (semicolon-separated)

Verification note. The UCI page states 4,424 instances, 36 features, a three-class target and no missing values. The file used in this study was loaded with pandas and these statements were checked on the file itself (Section 2.3). Because the UCI website could not be reached from the environment used for the analysis, the CSV was obtained from public GitHub copies of the same dataset; two independent copies were compared and found identical (4,424 × 37). Anyone re-running this project can download the original file from the UCI link above and place it at data/data.csv.

2.2 Why This Dataset Was Chosen

- **More than 15 features:** it has 36 input fields plus a target, which satisfies the assignment requirement.
- **Real-world relevance:** it describes real students and addresses a genuine problem – dropout and academic success.
- **Mixed data types:** continuous marks, counts, binary flags, coded categories and macro-economic indicators, so many chart types and statistical tests are meaningful.
- **Size:** 4,424 rows is enough for a stable train/validation/test split and for a small neural network, yet light enough to run on a laptop.
- **Clean but not trivial:** no missing values, but strong class imbalance, skewed variables, many outliers and several highly correlated features give real EDA work to do.
- **Supervised target:** the target is a three-class label, so this is a **multi-class classification** task (not regression).

2.3 Verified Structure of the Dataset

Table 2: Verified facts about the loaded file

Property	Value (measured on data/data.csv)
Rows (students)	4,424
Columns	37 (36 features + 1 target)
Missing cells	0
Duplicate rows	0

Property	Value (measured on data/data.csv)
Numeric features (continuous / count)	19
Binary flags	8
Nominal codes	9
Stored data types	int64: 29, float64: 7, text (target): 1
Target column	Target – Graduate / Dropout / Enrolled
Class counts	Graduate 2,209 (49.93%), Dropout 1,421 (32.12%), Enrolled 794 (17.95%)

2.4 Description of the Fields

Categorical fields in this dataset are stored as integer codes. Their meaning of individual code values is given in the UCI documentation; this report treats them as nominal labels and does not interpret the numbers themselves.

Table 3: All 36 input fields and the target

#	Field	Group	Type	Meaning
1	Marital status	Nominal	int	Code for marital status
2	Application mode	Nominal	int	Code for the way the student applied
3	Application order	Numeric	int	Order of choice in which the course was applied for
4	Course	Nominal	int	Code of the degree programme
5	Daytime/evening attendance	Binary	int	Daytime or evening classes
6	Previous qualification	Nominal	int	Code of qualification held before enrolment
7	Previous qualification (grade)	Numeric	float	Grade of previous qualification
8	Nacionality	Nominal	int	Code of nationality (spelling as in source)
9	Mother's qualification	Nominal	int	Code of mother's education level
10	Father's qualification	Nominal	int	Code of father's education level
11	Mother's occupation	Nominal	int	Code of mother's occupation
12	Father's occupation	Nominal	int	Code of father's occupation
13	Admission grade	Numeric	float	Grade at admission
14	Displaced	Binary	int	Student is displaced from home
15	Educational special needs	Binary	int	Has special educational needs
16	Debtor	Binary	int	Has unpaid debts to the institution
17	Tuition fees up to date	Binary	int	Tuition fees are paid up to date
18	Gender	Binary	int	Gender (binary code)
19	Scholarship holder	Binary	int	Holds a scholarship
20	Age at enrollment	Numeric	int	Age (years) at enrolment
21	International	Binary	int	International student
22	Curricular units 1st sem (credited)	Numeric	int	Units credited in 1st semester
23	Curricular units 1st sem (enrolled)	Numeric	int	Units enrolled in in 1st semester
24	Curricular units 1st sem (evaluations)	Numeric	int	Number of evaluations taken in 1st semester
25	Curricular units 1st sem (approved)	Numeric	int	Units passed in 1st semester
26	Curricular units 1st sem (grade)	Numeric	float	Average grade (0–20) in 1st semester
27	Curricular units 1st sem (without evaluations)	Numeric	int	Units with no evaluation in 1st semester
28	Curricular units 2nd sem (credited)	Numeric	int	Units credited in 2nd semester

#	Field	Group	Type	Meaning
29	Curricular units 2nd sem (enrolled)	Numeric	int	Units enrolled in in 2nd semester
30	Curricular units 2nd sem (evaluations)	Numeric	int	Number of evaluations taken in 2nd semester
31	Curricular units 2nd sem (approved)	Numeric	int	Units passed in 2nd semester
32	Curricular units 2nd sem (grade)	Numeric	float	Average grade (0–20) in 2nd semester
33	Curricular units 2nd sem (without evaluations)	Numeric	int	Units with no evaluation in 2nd semester
34	Unemployment rate	Numeric	float	National unemployment rate (%)
35	Inflation rate	Numeric	float	National inflation rate (%)
36	GDP	Numeric	float	Gross domestic product indicator
37	Target	Target	text	Dropout / Enrolled / Graduate at end of normal course duration

2.5 Basic Descriptive Statistics

Table 4 reports the mean, median, mode, standard deviation, quartiles and skewness of the 19 numeric features, computed on all 4,424 rows. Binary and nominal fields are summarised through frequency tables in Chapter 5.

Table 4: Descriptive statistics of numeric features

Feature	Mean	Median	Mode	Std	Min	Q1	Q3	Max	Skew
Application order	1.73	1.00	1.00	1.31	0.00	1.00	2.00	9.00	1.88
Previous qualification (grade)	132.61	133.10	133.10	13.19	95.00	125.00	140.00	190.00	0.31
Admission grade	126.98	126.10	130.00	14.48	95.00	117.90	134.80	190.00	0.53
Age at enrollment	23.27	20.00	18.00	7.59	17.00	19.00	25.00	70.00	2.06
CU 1st sem (credited)	0.71	0.00	0.00	2.36	0.00	0.00	0.00	20.00	4.17
CU 1st sem (enrolled)	6.27	6.00	6.00	2.48	0.00	5.00	7.00	26.00	1.62
CU 1st sem (evaluations)	8.30	8.00	8.00	4.18	0.00	6.00	10.00	45.00	0.98
CU 1st sem (approved)	4.71	5.00	6.00	3.09	0.00	3.00	6.00	26.00	0.77
CU 1st sem (grade)	10.64	12.29	0.00	4.84	0.00	11.00	13.40	18.88	-1.57
CU 1st sem (without evaluations)	0.14	0.00	0.00	0.69	0.00	0.00	0.00	12.00	8.21
CU 2nd sem (credited)	0.54	0.00	0.00	1.92	0.00	0.00	0.00	19.00	4.63
CU 2nd sem (enrolled)	6.23	6.00	6.00	2.20	0.00	5.00	7.00	23.00	0.79
CU 2nd sem (evaluations)	8.06	8.00	8.00	3.95	0.00	6.00	10.00	33.00	0.34
CU 2nd sem (approved)	4.44	5.00	6.00	3.02	0.00	2.00	6.00	20.00	0.31
CU 2nd sem (grade)	10.23	12.20	0.00	5.21	0.00	10.75	13.33	18.57	-1.31
CU 2nd sem (without evaluations)	0.15	0.00	0.00	0.75	0.00	0.00	0.00	12.00	7.27
Unemployment rate	11.57	11.10	7.60	2.66	7.60	9.40	13.90	16.20	0.21
Inflation rate	1.23	1.40	1.40	1.38	-0.80	0.30	2.60	3.70	0.25
GDP	0.00	0.32	0.32	2.27	-4.06	-1.70	1.79	3.51	-0.39

3. Exploratory Data Analysis and Preprocessing

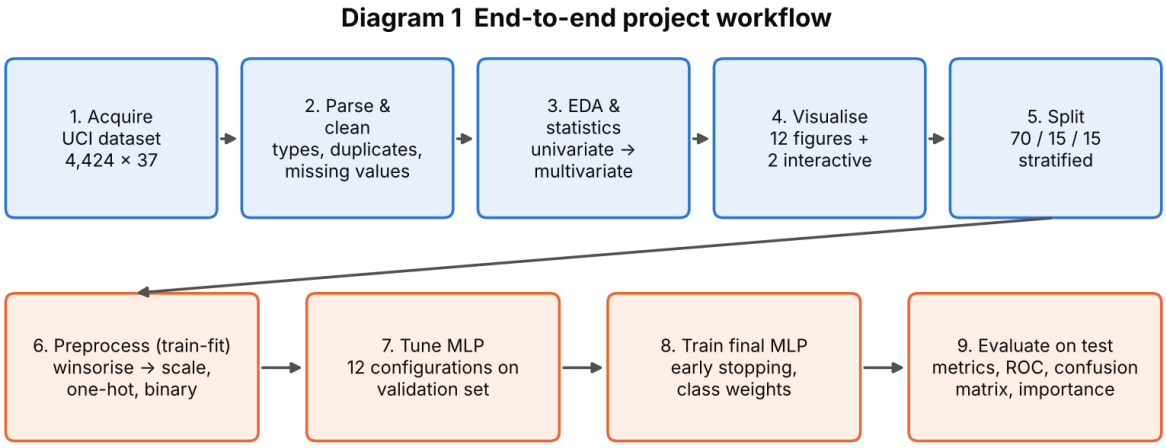


Diagram 1: End-to-end project workflow

Diagram 1 shows the order of work. The top row (steps 1–5) is data understanding and visualisation; the bottom row (steps 6–9) is modelling. Every step is a separate Python module in the GitHub repository.

3.1 Loading and Inspecting the Data

The CSV is loaded with pandas (pd.read_csv with sep=';'). The header contained stray white-space, which was stripped so that column names can be used safely in code. The shape is 4,424 rows × 37 columns. df.info(), df.describe() and value counts were used to check data types and ranges.

3.2 Missing Values and Duplicates

Missing cells found: **0**. Duplicate rows found: **0**. The cleaning function therefore removed no rows (4,424 rows kept). The code still includes imputation-free checks so the same script is safe on a new data release.

A hidden pattern that looks like missing data. The average-grade fields contain many zeros. Checking the data shows that the average grade of a semester is exactly 0 **if and only if** no curricular unit was approved in that semester (this held for 100% of rows in both semesters). So a zero is a real, meaningful value ("nothing passed"), not an error, and was not imputed. 16.2% of students had no approved unit in semester 1 and 19.7% in semester 2; of those students, 79.4% and 83.6% respectively finally dropped out.

3.3 Outlier Identification and Treatment

Outliers were identified with the IQR rule (values outside Q1 − 1.5 × IQR or Q3 + 1.5 × IQR) and the |z| > 3 rule for the 14 numeric features with more than 10 distinct values (Table 5, Figure 4).

Table 5: Outlier summary (eight features with the highest IQR outlier share)

Feature	Lower fence	Upper fence	IQR outliers	Share	|z|>3
CU 2nd sem (grade)	6.88	17.21	877	19.8%	0
CU 1st sem (grade)	7.40	17.00	726	16.4%	0
CU 1st sem (credited)	0.00	0.00	577	13.0%	149
CU 2nd sem (credited)	0.00	0.00	530	12.0%	110
Age at enrollment	10.00	34.00	441	10.0%	101

Feature	Lower fence	Upper fence	IQR outliers	Share	|z|>3
CU 1st sem (enrolled)	2.00	10.00	424	9.6%	106
CU 2nd sem (enrolled)	2.00	10.00	369	8.3%	82
CU 1st sem (without evaluations)	0.00	0.00	294	6.6%	62

Decision. The extreme values are real students (for example mature students aged up to 70, or students who passed many units), and zero grades are meaningful, so rows were **not deleted**. Instead, inside the model pipeline each numeric feature is *winsorised*: values below the 1st or above the 99th percentile (learned from the training set only) are clipped to those limits. Because more than 1% of students have a zero grade, the 1st percentile of the grade fields is itself 0, so the meaningful zeros are preserved.

3.4 Cleaning, Transformation and Encoding

Table 6: Preprocessing applied to each group of features

Group	Features	Treatment	Reason
Numeric	19 fields (grades, counts, ages, macro-economic)	Winsorise 1–99 % → StandardScaler (mean 0, std 1)	Neural networks train poorly when inputs have very different scales
Nominal codes	9 fields (course, application mode, parents' education/occupation, …)	One-hot encoding; categories with < 2 % of training rows are grouped as 'infrequent'	Codes have no order, so they must not be treated as numbers; grouping avoids hundreds of sparse columns
Binary flags	8 fields (Debtor, Gender, Scholarship holder, …)	Used as 0/1	Already numeric and unordered
Target	Target	Label-encoded: Dropout = 0, Enrolled = 1, Graduate = 2	Required by the softmax output layer

After encoding, the 36 raw features become **98 model input columns**.

3.5 Train / Validation / Test Split and Leakage Prevention

Table 7: Data split (stratified, random_state = 42)

Subset	Rows	Share	Purpose
Training	3,096	70%	Fit the scaler/encoder and learn the network weights
Validation	664	15%	Choose hyperparameters and stop training early
Test	664	15%	Final, one-time unbiased evaluation

Preventing data leakage. (1) The split is done *before* any statistic is learned. (2) The winsoriser, scaler and one-hot encoder are fitted on the training rows only and then applied unchanged to validation and test rows. (3) The test set is used only once, after all tuning decisions are final. (4) The split is stratified, so each subset keeps the same Dropout/Enrolled/Graduate proportions.

3.6 Key Findings from EDA

- The target is moderately imbalanced: Graduate 49.93%, Dropout 32.12%, Enrolled 17.95%. Always predicting 'Graduate' would already give about 50% accuracy, so accuracy alone is not enough; macro-F1 and class weights are used.
- Dropout rate is 86.55% for students whose tuition fees are **not** up to date versus 24.74% for those who are up to date.

- Academic performance in the first two semesters (units approved and average grade) separates the three outcomes much more strongly than admission grade does.
- Macro-economic indicators (unemployment, inflation, GDP) are almost uncorrelated with academic variables (Section 5.3).
- Several semester-1 and semester-2 variables are almost duplicates of each other (Spearman ρ up to 0.96), so the data has strong multicollinearity.

4. Data Visualization

4.1 The Seven Stages of Visualization

The assignment sheet asks that the visualisations follow "all 7 stages of visualization" but does not list them. This report therefore states the framework it uses: the seven-stage visualisation process of Ben Fry (2007), *Visualizing Data* (O'Reilly): **Acquire, Parse, Filter, Mine, Represent, Refine, Interact**. Table 8 maps each stage to the work done in this project.

Table 8: Seven stages of visualization applied to the student dataset

Stage	Meaning	How it was applied here	Evidence
1. Acquire	Obtain the data	Selected the UCI dataset (id 697), checked licence, rows and columns, and loaded data/data.csv	Sections 2.1, 2.3
2. Parse	Give the data structure and meaning	Fixed delimiter and header, assigned numeric / binary / nominal roles, encoded the target	Sections 2.4, 3.1, 3.4
3. Filter	Remove what is not needed	Duplicate and missing-value checks; selected key variables for each chart; courses with ≥ 100 students in Fig. 12	Sections 3.2, 3.3
4. Mine	Find patterns with statistics	Descriptive statistics, Kruskal–Wallis and chi-square tests, Spearman correlation, VIF, PCA, distribution fitting	Chapter 5
5. Represent	Choose a basic visual form	Bar, histogram, boxplot, scatter, heat-map, pairplot, Q–Q plot, PCA projection – matched to the data type	Figures 1–12
6. Refine	Improve clarity and design	Colour-blind-safe palette checked with a validator, same colour per class in every chart, direct value labels, mean/median lines, transparency and jitter against over-plotting, sorted bars	All figures
7. Interact	Let the viewer explore	Two Plotly pages with hover, zoom and legend filtering: outputs/eda/interactive_*.html*	Section 4.3

4.2 Visualization Design Choices (Refine stage)

A single colour is reserved for each outcome in every figure: **blue = Graduate, orange = Dropout, green = Enrolled**. These three hues come from a palette whose adjacent colours were checked for colour-vision-deficiency separation. Grid-lines are light, axes are labelled with units, titles are numbered, and no chart uses a dual axis or a rainbow scale.

4.3 Interactive Visualizations

Two interactive HTML pages are saved in outputs/eda/: **interactive_grades_scatter.html** (semester-1 vs semester-2 grade; hover shows age, admission grade, course and fee status; clicking a legend entry hides or shows a class) and **interactive_course_outcomes.html** (stacked counts of outcomes for each course code). They can be opened in any web browser and complete stage 7.

4.4 Visualization Gallery with Explanations

Each figure below is followed by a table that states its purpose, the variables used, the chart type, why it was selected and the insight obtained, as the assignment requires. Figures 1–12 are EDA figures; model figures are in Chapter 7.

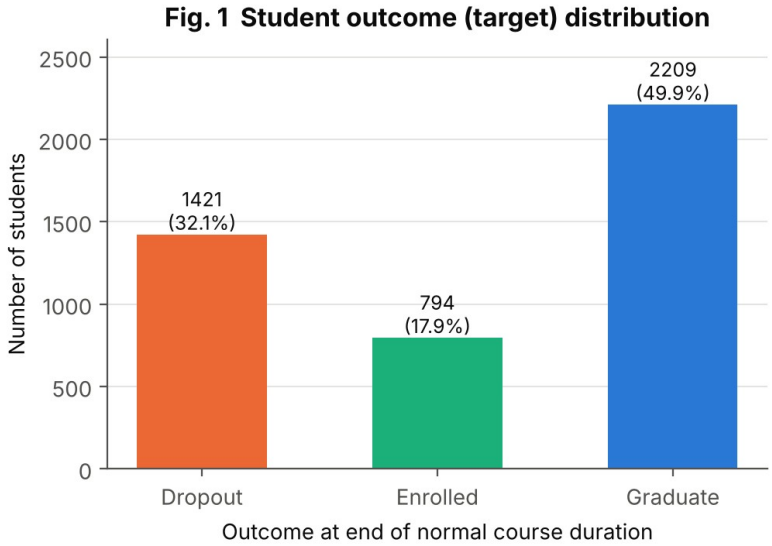


Figure 1: Student outcome distribution

Aspect	Explanation
Purpose	Show how many students fall into each outcome class.
Variables	Target
Chart type	Bar chart with count and percentage labels
Why selected	Class balance decides which metrics and training tricks are needed; a bar chart is the clearest form for a few categories.
Insight	Graduate 2,209 (49.93%), Dropout 1,421 (32.12%), Enrolled 794 (17.95%). Enrolled is the smallest class, so it is expected to be hardest to predict; class weights and macro-F1 are used.

Fig. 2 Histograms of key numeric variables

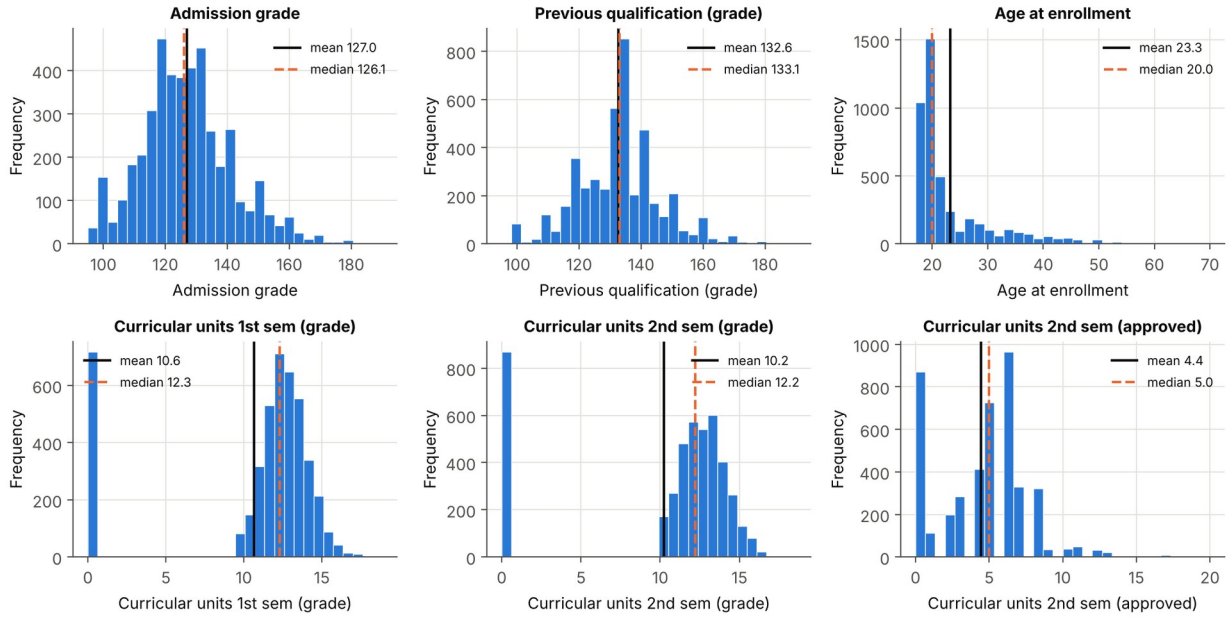


Figure 2: Histograms of key numeric variables

Aspect	Explanation
Purpose	Show the shape, centre and spread of six important numeric variables.
Variables	Admission grade, Previous qualification (grade), Age at enrollment, CU 1st/2nd sem (grade), CU 2nd sem (approved)
Chart type	Histograms (30 bins) with mean (solid line) and median (dashed line)
Why selected	Histograms are the standard univariate view of a continuous variable; the mean–median gap shows skewness at a glance.
Insight	Admission grade is roughly bell-shaped (mean 127.0, median 126.1). Age is strongly right-skewed (skew 2.06; median 20, mean 23.3) – most students enrol at 18–20 but a tail of mature students reaches 70. The semester grades are left-skewed with a spike at 0 (median 12.3 vs mean 10.6), caused by students who passed no unit.

Fig. 3 Boxplots of key variables by student outcome

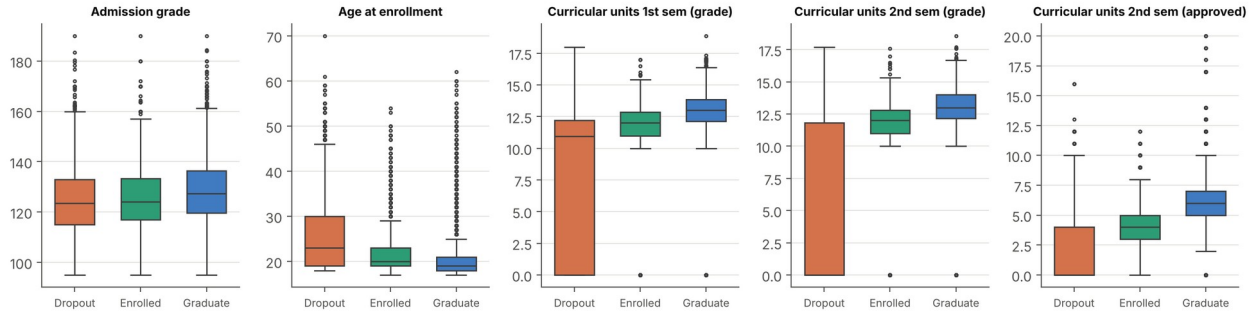


Figure 3: Boxplots of key variables by outcome

Aspect	Explanation
Purpose	Compare the distribution of important variables across the three outcome classes and reveal outliers.
Variables	Target vs Admission grade, Age at enrollment, CU 1st/2nd sem (grade), CU 2nd sem (approved)
Chart type	Grouped boxplots
Why selected	Boxplots compare medians, spread and outliers between groups compactly; they are the natural bivariate view for numeric-vs-category.
Insight	Median 2nd-semester units approved rise from 0 (Dropout) to 4 (Enrolled) to 6 (Graduate). Dropouts are older (median age 23 vs 19). Admission-grade medians differ only a little (123.6, 124.1, 127.4), so entry marks say less about the outcome than later performance does.

Fig. 4 Share of IQR outliers per numeric feature

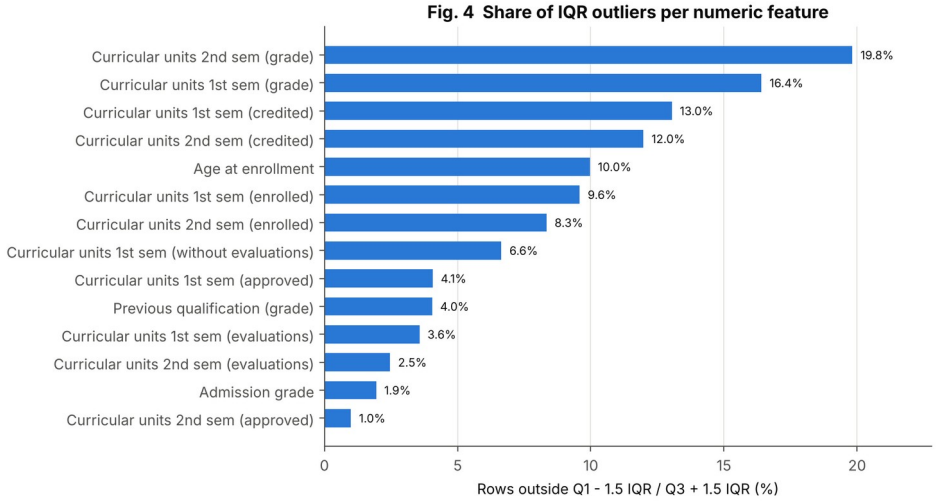


Figure 4: Share of IQR outliers per numeric feature

Aspect	Explanation
Purpose	Quantify how many rows are flagged as outliers in each numeric feature.
Variables	14 numeric features with more than 10 distinct values
Chart type	Sorted horizontal bar chart (percentage labels)
Why selected	A ranked bar chart makes it easy to see which features need a treatment decision.
Insight	The semester-grade fields flag the most rows (19.8% and 16.4%) mainly because of the zero grades described in Section 3.2, not because of data errors. Age flags 10.0% (mature students). Since these are genuine values, they were winsorised in the pipeline rather than deleted.

Fig. 5 Continuous distribution: Admission grade

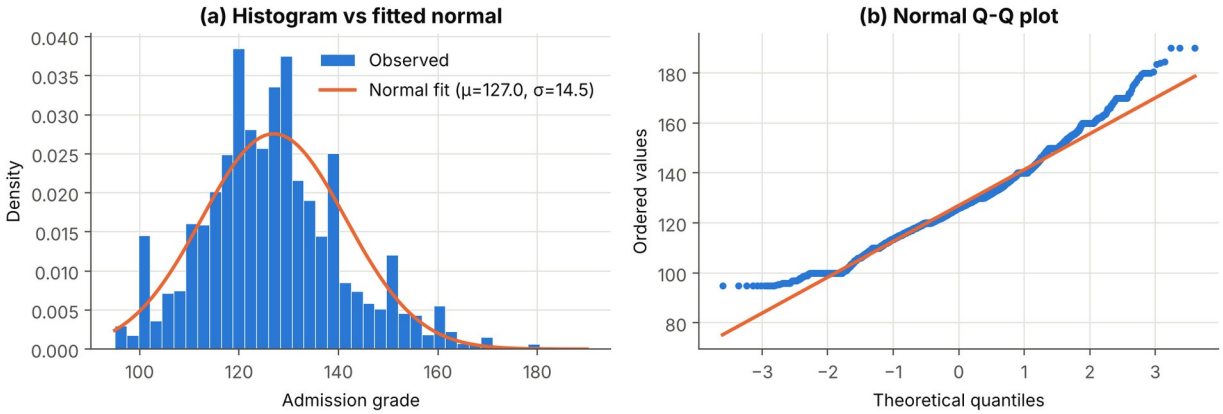


Figure 5: Continuous distribution: Admission grade

Aspect	Explanation
Purpose	Check whether admission grade follows a normal distribution.
Variables	Admission grade
Chart type	Histogram with fitted normal curve, and normal Q–Q plot
Why selected	The histogram shows the fit visually; the Q–Q plot is more sensitive to tail behaviour.
Insight	Fitted normal: mean 127.0, SD 14.5. Skewness 0.53, excess kurtosis 0.66. D'Agostino's test rejects exact normality ($p < 0.001$; with 4,424 rows even small departures are detected), and the Q–Q plot shows why: the lower tail is cut off at the minimum grade of 95 and the upper tail is heavier than normal. The distribution is bell-shaped but right-skewed.

Fig. 6 Discrete distributions: Poisson and Binomial

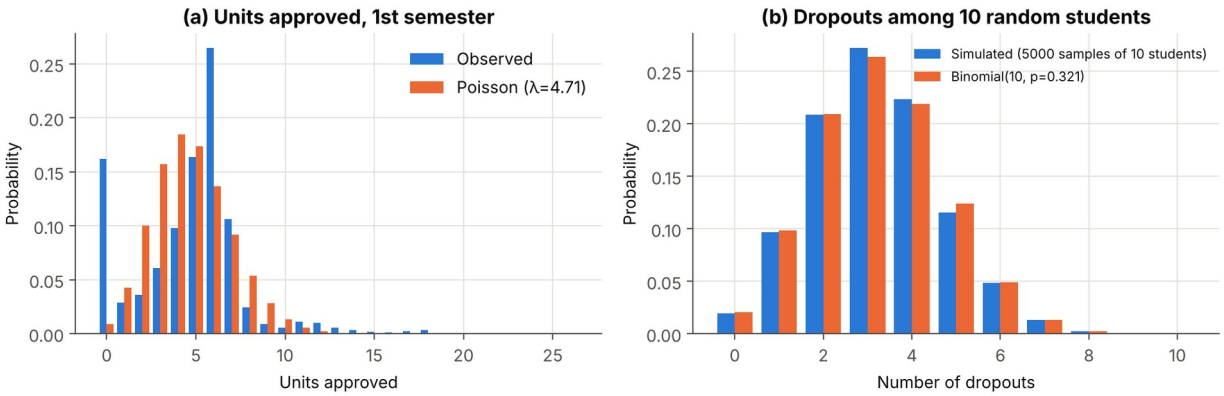


Figure 6: Discrete distributions: Poisson and Binomial

Aspect	Explanation
Purpose	Test whether a count variable fits a Poisson model and illustrate a Binomial model for the number of dropouts.
Variables	(a) CU 1st sem (approved); (b) Target = Dropout (yes/no)
Chart type	Side-by-side bar charts of observed vs theoretical probabilities
Why selected	Observed-vs-theoretical bars show directly where a model fits or fails.
Insight	(a) A Poisson with the same mean (λ = 4.71) fits badly: the variance-to-mean ratio is 2.03 (Poisson would be 1) and 16.2% of students have 0 units approved versus 0.9% expected – the data are over-dispersed and zero-inflated. (b) Dropout is a Bernoulli variable with p = 0.321 (95% Wilson interval 0.308–0.335). The number of dropouts among 10 random students follows Binomial(10, 0.321); a simulation of 5,000 random samples matches it closely. For example, P(at least 5 dropouts among 10) = 0.189.

Fig. 7 Spearman correlation heatmap (numeric features)

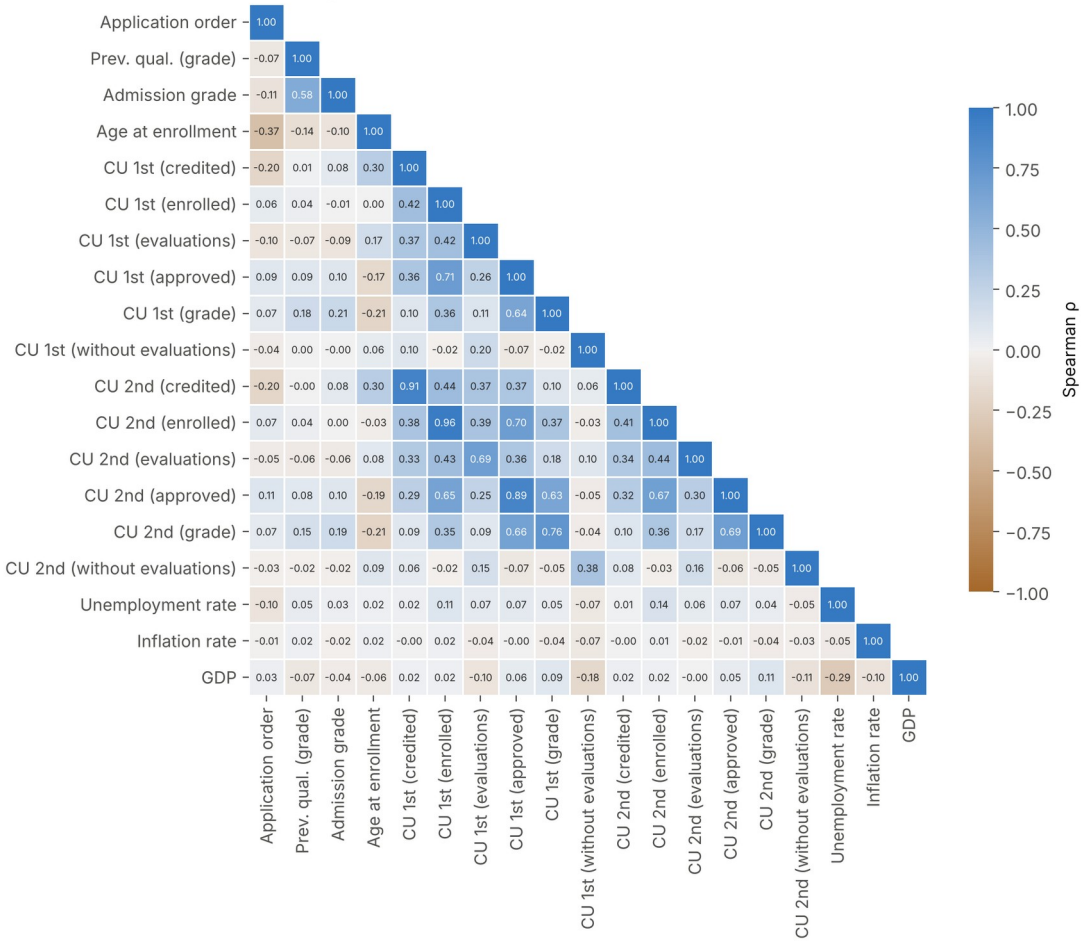


Figure 7: Spearman correlation heatmap

Aspect	Explanation
Purpose	Show monotonic relationships among all 19 numeric features.
Variables	All numeric features
Chart type	Lower-triangle heat-map with diverging colour scale (neutral midpoint at 0) and printed coefficients
Why selected	A heat-map is the standard multivariate summary; Spearman is used because many variables are skewed or counts.
Insight	Strongest pairs: CU enrolled 1st vs 2nd semester ρ = 0.962, credited ρ = 0.914, approved ρ = 0.892, grade ρ = 0.762. Admission grade and previous-qualification grade correlate at 0.58. Unemployment, inflation and GDP have |ρ| below 0.3 with every other variable. Highly correlated groups produce VIF values above 10 (Section 5.3), so the network is regularised.

Fig. 8 Outcome mix by binary factors

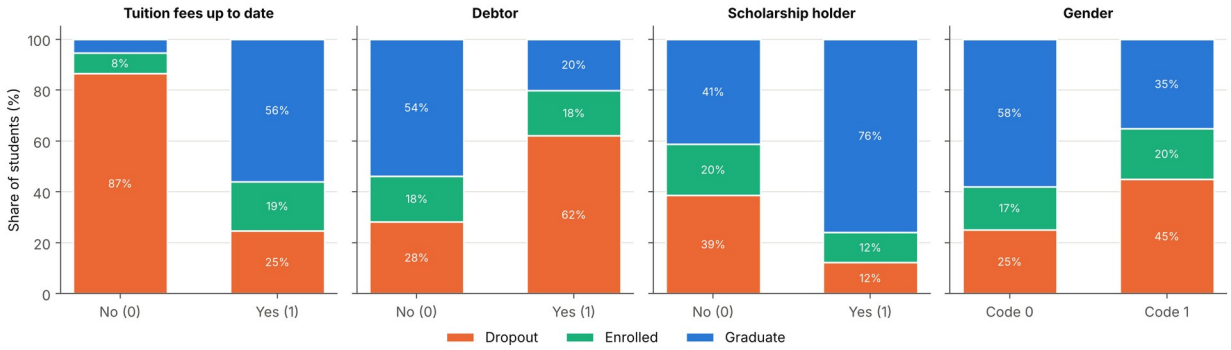


Figure 8: Outcome mix by binary factors

Aspect	Explanation
Purpose	Compare the outcome mix between the two values of four binary factors.
Variables	Tuition fees up to date, Debtor, Scholarship holder, Gender vs Target
Chart type	100% stacked bar charts
Why selected	Percent-stacked bars compare proportions fairly even when group sizes differ.
Insight	Not-up-to-date tuition: 86.55% dropout vs 24.74%. Debtors: 62.03% vs 28.28%. Scholarship holders: 12.19% vs 38.71%. Gender code 1: 45.05% vs code 0: 25.1%. These are associations, not causes – for instance unpaid fees may be a consequence of leaving as well as a reason for it.



Figure 9: First vs second semester grade

Aspect	Explanation
Purpose	Show how first- and second-semester performance relate and how outcomes are spread over them.
Variables	CU 1st sem (grade), CU 2nd sem (grade), Target
Chart type	Scatter plot (colour = outcome, transparency + small jitter)
Why selected	Scatter plots are the standard bivariate view of two continuous variables; colour adds the third (categorical) dimension.
Insight	The main cloud shows a clear positive relationship (ρ = 0.76) with graduates (blue) concentrated at higher grades. There are no points between 0 and about 10 because a semester grade is zero exactly when no unit was passed. The dense dot at (0, 0) and the strips along the axes are almost entirely dropouts – students who passed nothing in one or both semesters.

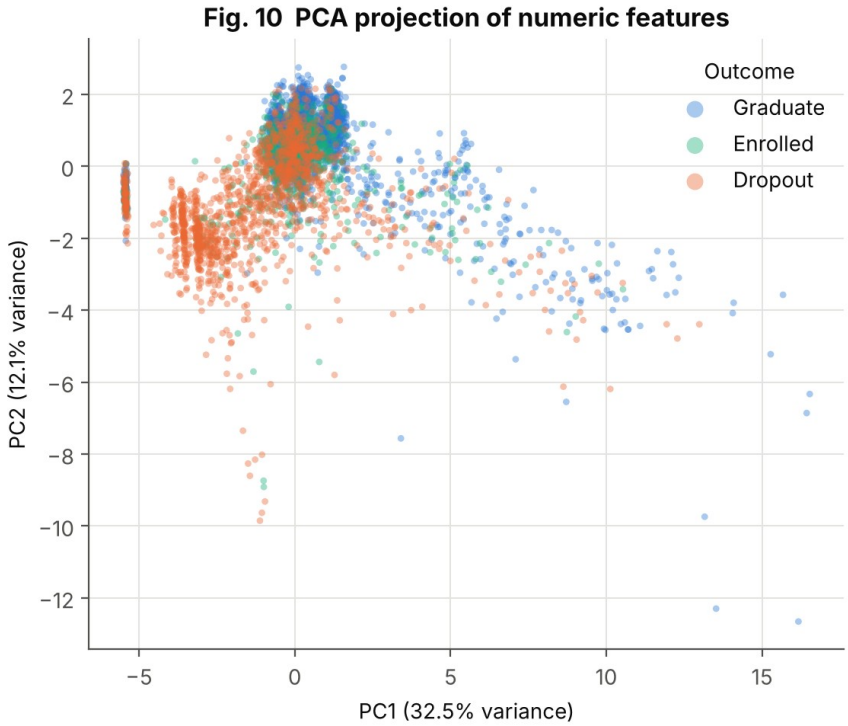


Figure 10: PCA projection of numeric features

Aspect	Explanation
Purpose	See whether the classes separate when the 19 numeric features are compressed into two dimensions.
Variables	All numeric features (standardised), Target
Chart type	PCA scatter plot
Why selected	PCA is the usual multivariate technique to visualise high-dimensional data in 2-D.
Insight	PC1 explains 32.5% and PC2 12.1% of the variance; the first five components together explain 68.3%. Dropouts (orange) lie mostly to the left, graduates to the right/upper area, but there is heavy overlap and Enrolled students (green) sit in between – a visual explanation of why Enrolled is the hardest class to predict.

Fig. 11 Pairplot of key variables

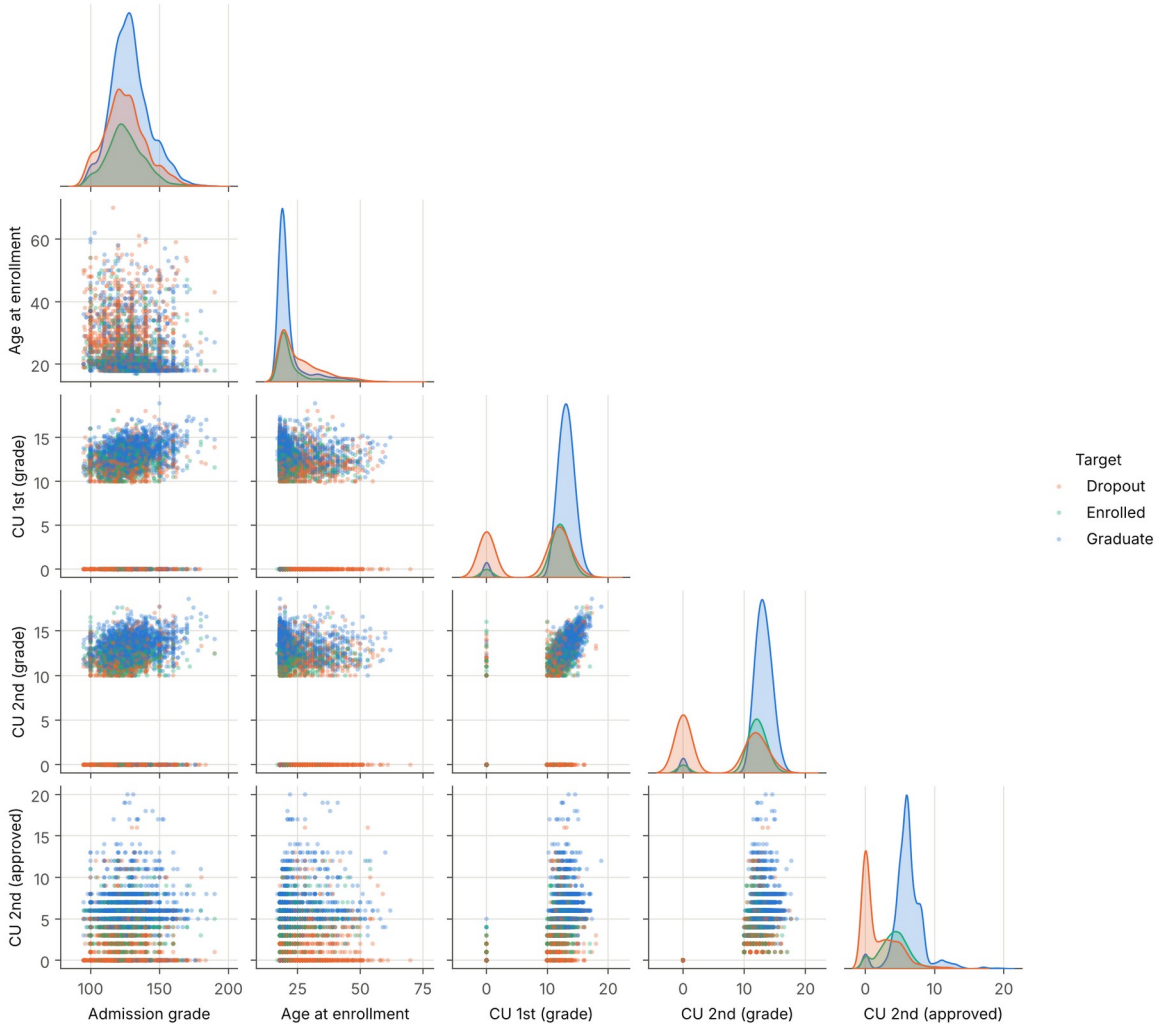


Figure 11: Pairplot of key variables

Aspect	Explanation
Purpose	Show all pairwise relationships and each variable's distribution for five key variables in one view.
Variables	Admission grade, Age, CU 1st/2nd sem (grade), CU 2nd sem (approved), Target
Chart type	Pairplot (scatter matrix with KDE diagonals)
Why selected	A pairplot is the quickest way to compare many pairs and to see class-wise distributions on the diagonal.
Insight	Grade and approved-unit variables are positively related to each other, and the dropout curves on the diagonals have a separate peak at zero. Age shows dropouts with a longer right tail. Admission grade barely separates the classes.

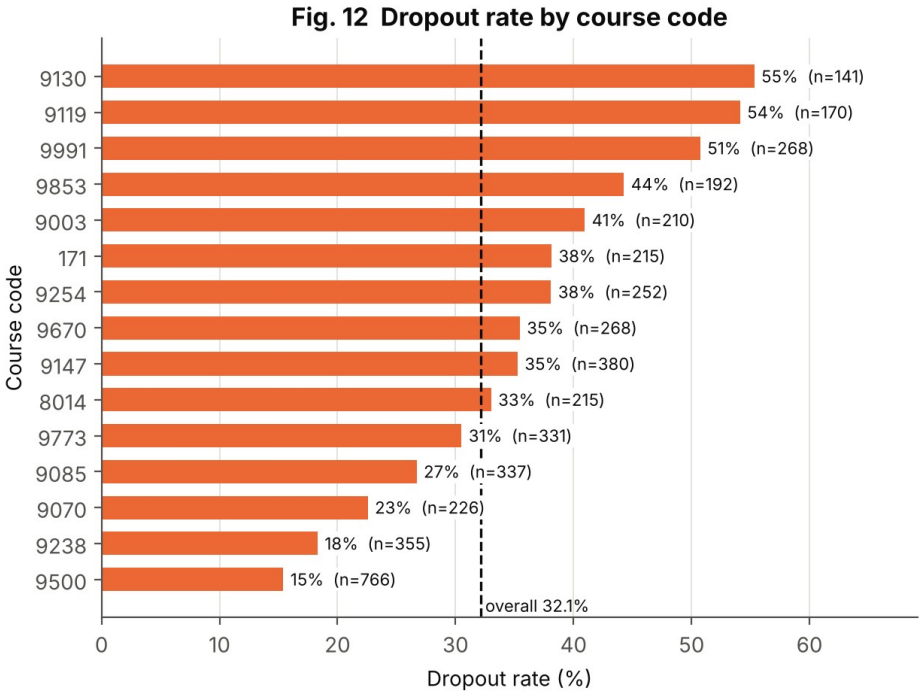


Figure 12: Dropout rate by course code

Aspect	Explanation
Purpose	Compare dropout rates between degree programmes with enough students.
Variables	Course (code), Target
Chart type	Sorted horizontal bar chart with overall-rate reference line
Why selected	Sorting and a reference line make differences between programmes easy to rank; only courses with ≥ 100 students are shown so that rates are stable.
Insight	Among the 15 courses with at least 100 students, dropout rates range from 15.4% (course 9500, n = 766) to 55.3% (course 9130, n = 141), compared with 32.1% overall. Course is therefore an informative predictor; code meanings are listed in the UCI documentation.

5. Statistical and Distribution-Based Analysis

5.1 Univariate Analysis

Table 4 (Section 2.5) gives the mean, median, mode, standard deviation and quartiles of every numeric feature. The main points are:

- **Centre and spread:** admission grade mean 127.0 (SD 14.5); age mean 23.3 but median 20 and mode 18.
- **Skewness:** age has the strongest positive skew (2.06), so the median describes a typical student better than the mean. Grade fields are left-skewed because of zeros.
- **Quantiles:** half of all students are aged 20 or less (median), 95% are 41 or younger, while the maximum is 70.
- **Categorical fields:** the outcome classes are 49.9% / 32.1% / 18.0%; the binary flags are unbalanced (for example only a minority are debtors or scholarship holders), which is reflected in the stratified split.

5.2 Probability Distributions

Table 9: Distribution analysis results

Variable	Model	Result	Conclusion
Admission grade (continuous)	Normal	μ = 127.0, σ = 14.5; KS D = 0.066; D'Agostino p < 0.001	Approximately bell-shaped but not exactly normal (skew 0.53, truncated at 95)
Previous qualification (grade)	Normal	μ = 132.6, σ = 13.2; KS D = 0.091; p < 0.001	Near-symmetric (skew 0.31), heavier tails than normal
Age at enrollment (continuous)	Normal vs log-normal	KS D = 0.267 (normal) vs 0.242 (log-normal)	Both poor; log-normal slightly better for the right-skewed shape
Dropout (discrete, yes/no)	Bernoulli / Binomial(10, p)	p = 0.321; Wilson 95% CI 0.308–0.335	Binomial pmf matches simulated samples (Fig. 6b)
CU 1st sem approved (count)	Poisson	mean 4.71, variance 9.57, ratio 2.03	Rejected: over-dispersed with excess zeros
CU 2nd sem approved (count)	Poisson	mean 4.44, variance 9.09, ratio 2.05	Rejected: over-dispersed
CU 1st sem without evaluations (count)	Poisson	mean 0.14, ratio 3.47, zeros 93.4%	Rejected: very zero-heavy

In simple words: a *Poisson* distribution describes counts of rare, independent events and has variance equal to its mean. Here the variance is about twice the mean because there are two kinds of student – those who pass nothing and those who pass most units – so a single Poisson cannot fit; a mixture or zero-inflated model would be more suitable.

5.3 Bivariate Analysis

Every feature was tested against the three-class outcome. Numeric features used the **Kruskal–Wallis H test** (does not assume normality) with the effect size ε^2; binary and nominal features used the **chi-square test of independence** with Cramér's V. With 4,424 students almost every p-value is below 0.001, so the *effect size* is what ranks the features (Table 10).

Table 10: Twelve features most strongly associated with the outcome

Feature	Type	Test	Statistic	p-value	Effect size
Tuition fees up to date	binary	Chi-square	823.6	< 0.001	0.431
Curricular units 2nd sem (approved)	numeric	Kruskal-Wallis	1892.3	< 0.001	0.428
Curricular units 1st sem (approved)	numeric	Kruskal-Wallis	1561.7	< 0.001	0.353
Curricular units 2nd sem (grade)	numeric	Kruskal-Wallis	1390.7	< 0.001	0.314
Scholarship holder	binary	Chi-square	409.9	< 0.001	0.304
Course	nominal	Chi-square	558.3	< 0.001	0.251
Curricular units 1st sem (grade)	numeric	Kruskal-Wallis	1094.2	< 0.001	0.247
Debtor	binary	Chi-square	259.3	< 0.001	0.242
Gender	binary	Chi-square	233.3	< 0.001	0.230
Application mode	nominal	Chi-square	466.5	< 0.001	0.230
Mother's occupation	nominal	Chi-square	291.9	< 0.001	0.182
Father's occupation	nominal	Chi-square	264.5	< 0.001	0.173

Interpretation. Tuition-fee status (V = 0.43) and units approved in semester 2 ($\varepsilon^2 = 0.43$) have the largest effects, followed by units approved in semester 1, semester-2 grade, scholarship status and course.

Admission grade has a small effect ($\varepsilon^2 = 0.016$), and macro-economic indicators are weakest. The tests show *association*, not causation.

5.4 Multivariate Analysis

- **Correlation structure (Fig. 7):** the strongest correlations are between the same measure in consecutive semesters (ρ between 0.76 and 0.962).
- **Multicollinearity:** variance inflation factors above 10 were found for CU 1st sem (enrolled) (23.5), CU 2nd sem (enrolled) (16.5), CU 1st sem (credited) (15.6), CU 1st sem (approved) (12.6), CU 2nd sem (credited) (12.4), CU 2nd sem (approved) (10.2). Redundant inputs do not harm a neural network's accuracy as much as a linear regression's coefficients, but they make individual feature effects harder to interpret.
- **PCA (Fig. 10):** five components explain 68.3% of the variance of the 19 numeric features, showing that the numeric information is partly redundant.
- **Combined effects (Figs. 8, 9, 11):** low performance, unpaid fees and absence of a scholarship tend to occur together in the dropout group.

5.5 Summary of Statistical Findings

Table 11: Statistical findings in plain words

Finding	Evidence
Early academic performance is the strongest signal of the final outcome	Largest Kruskal–Wallis effects ($\varepsilon^2 = 0.43$ and 0.35); Fig. 3, 9
Financial factors matter	Dropout 86.55% vs 24.74% by fee status; scholarship holders 12.19% vs 38.71%
Entry marks are weak predictors	Admission-grade $\varepsilon^2 = 0.016$; median grades 123.6 / 124.1 / 127.4
Macro-economic indicators add little	$\lvert\rho\rvert < 0.3$ with all variables; smallest effect sizes
Counts are not Poisson	Dispersion index about 2–3.5; excess zeros

6. Deep Learning Model

6.1 Why a Multilayer Perceptron?

The data are in table form (numbers and codes), so the natural deep-learning model is a **multilayer perceptron (MLP)** – a stack of fully connected layers (Goodfellow et al., 2016). CNNs are designed for images and RNN/LSTM/Transformers for sequences or text, which this dataset does not contain. An MLP can learn non-linear combinations of features (for example 'low grades *and* unpaid fees') that a simple linear model cannot. To judge whether the extra complexity is worthwhile, logistic regression and a random forest were trained on the same data as baselines.

6.2 Inputs, Target and Pipeline

Table 12: Model inputs and output

Item	Description
Input	98 columns produced from 36 raw features (scaled numeric + one-hot nominal + binary)
Target	3 classes: Dropout (0), Enrolled (1), Graduate (2)
Task type	Multi-class classification

Item	Description
Pipeline	Winsorise (1–99 %) → standardise; one-hot with rare-category grouping; binary pass-through. Fitted on the training set only and saved as models/preprocessor.joblib

6.3 Architecture and Hyperparameters

Diagram 2 MLP architecture (24,003 parameters)

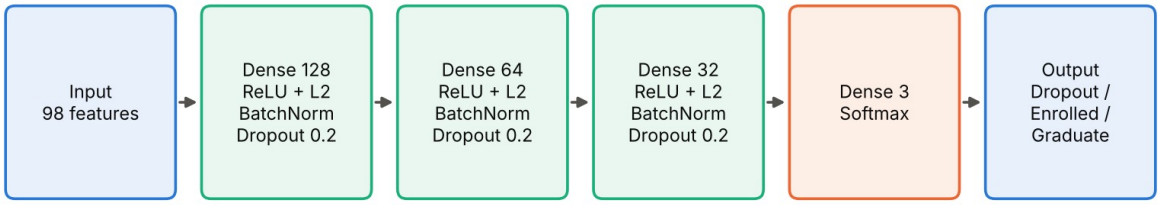


Diagram 2: MLP architecture of the final model

Table 13: Final model configuration

Component	Setting	Simple explanation
Hidden layers	128 → 64 → 32 neurons	Each layer learns more abstract combinations of the inputs
Activation	ReLU (hidden), Softmax (output)	ReLU adds non-linearity; softmax turns the 3 outputs into probabilities that sum to 1
Regularisation	Dropout 0.2, L2 weight penalty 1e-4, BatchNorm	Reduce over-fitting and stabilise training
Loss function	Sparse categorical cross-entropy	Penalises confident wrong probabilities
Optimizer	Adam, learning rate 0.001	Adaptive gradient method (Kingma & Ba, 2015)
Batch size / max epochs	64 / 100	Samples per weight update / upper limit on passes over the data
Early stopping	Monitor validation loss, patience 10, restore best weights	Training actually ran 23 epochs; best epoch 13
Class weights	Dropout 1.04, Enrolled 1.86, Graduate 0.67	Give the rarer 'Enrolled' class more influence on the loss
Parameters	24,003	Includes batch-normalisation statistics
Random seed	42	Makes the run reproducible

6.4 Training, Validation and Testing Procedure

1. Fit the preprocessing pipeline on the training rows and transform all three subsets.
2. For each hyperparameter configuration, train on the training set and monitor loss on the validation set; stop early when it no longer improves.
3. Choose the configuration with the best **validation macro-F1** (ties broken by validation loss).
4. Evaluate the chosen model **once** on the untouched test set and report accuracy, macro precision/recall/F1, ROC-AUC, confusion matrix and importance.

6.5 Basic Hyperparameter Tuning

A small grid search over **3 architectures × 2 dropout rates × 2 learning rates = 12 configurations** was run. All other settings were fixed. Selection used the validation set only.

Table 14: Hyperparameter search results (sorted by validation macro-F1)

Hidden layers	Dropout	Learning rate	Epochs	Val macro-F1	Val loss	Val accuracy
(128, 64, 32)	0.2	0.001	23	0.699	0.700	0.735
(128, 64)	0.4	0.001	30	0.687	0.695	0.721
(128, 64)	0.4	0.0003	65	0.686	0.694	0.718
(128, 64, 32)	0.4	0.0003	65	0.679	0.713	0.714
(128, 64)	0.2	0.0003	34	0.679	0.701	0.714
(128, 64, 32)	0.2	0.0003	36	0.672	0.717	0.703
(128, 64)	0.2	0.001	17	0.671	0.695	0.702
(64, 32)	0.4	0.001	38	0.671	0.666	0.714
(128, 64, 32)	0.4	0.001	30	0.670	0.706	0.703
(64, 32)	0.4	0.0003	85	0.668	0.681	0.708
(64, 32)	0.2	0.0003	40	0.652	0.705	0.696
(64, 32)	0.2	0.001	20	0.651	0.689	0.697

The best configuration was [128, 64, 32] with dropout 0.2 and learning rate 0.001 (validation macro-F1 0.699). The spread between the best and worst configuration is 0.048 macro-F1, so the model is fairly insensitive to these choices; differences of about 0.01–0.02 between neighbouring configurations are within the noise expected from a 664-row validation set. Figure 19 shows the same results as a chart.

6.6 Feature Analysis and Feature Importance

Two complementary methods were used. (1) **Statistical association** (Section 5.3) shows which features relate to the outcome in the raw data. (2) **Permutation importance** measures what the *trained model* relies on: each raw feature in the test set is randomly shuffled 10 times and the drop in macro-F1 is recorded; a large drop means the model depends on that feature. Because shuffling is done on the raw feature (before one-hot encoding), all columns that came from one original field are shuffled together.

Table 15: Top 10 features by permutation importance (test set)

Rank	Feature	Mean macro-F1 drop	Std (10 repeats)
1	CU 2nd sem (approved)	0.1824	0.0096
2	CU 1st sem (approved)	0.1177	0.0115
3	CU 2nd sem (grade)	0.0330	0.0066
4	Tuition fees up to date	0.0306	0.0102
5	Age at enrollment	0.0171	0.0061
6	CU 2nd sem (evaluations)	0.0153	0.0086
7	CU 1st sem (evaluations)	0.0138	0.0096
8	CU 2nd sem (credited)	0.0114	0.0043
9	Father's occupation	0.0106	0.0081
10	CU 2nd sem (enrolled)	0.0100	0.0048

The model relies overwhelmingly on units approved in semester 2 (drop 0.182) and semester 1 (drop 0.118), followed by semester-2 grade and tuition-fee status. This agrees with the statistical findings in Chapter 5.

Importance of correlated features is shared between them (for example approved units and grade), so a small value for one feature does not prove it is unrelated to the outcome.

7. Model Evaluation and Result Visualization

7.1 Evaluation Metrics

All numbers below were measured by running the code in the appendix (seed 42). The test set contains 664 students that were never used for fitting, scaling or tuning.

Table 16: MLP performance on validation and test sets

Metric	Validation	Test
Accuracy	0.735	0.756
Macro precision	0.711	0.719
Macro recall	0.713	0.723
Macro F1-score	0.699	0.715
Weighted F1-score	0.747	0.766
ROC-AUC (one-vs-rest, macro)	0.869	0.895

Test accuracy is 75.6% (approximate 95% Wilson interval 72.2%–78.7% for 664 students), compared with about 50% for always predicting 'Graduate'. Validation and test scores are close, which suggests the tuning did not over-fit the validation set badly.

Table 17: Per-class results on the test set

Class	Precision	Recall	F1-score	Support
Dropout	0.84	0.73	0.79	214
Enrolled	0.44	0.61	0.51	119
Graduate	0.87	0.82	0.84	331

7.2 Comparison with Baseline Models

Table 18: Test-set comparison of the MLP and baseline models (same split and preprocessing)

Model	Accuracy	Macro P	Macro R	Macro F1	ROC-AUC
MLP (proposed)	0.756	0.719	0.723	0.715	0.895
Logistic regression	0.767	0.735	0.745	0.732	0.902
Random forest	0.803	0.753	0.745	0.748	0.907

Honest reading of the comparison. On this dataset the MLP is *competitive with*, but **not better than**, logistic regression and a random forest; the random forest has the highest accuracy, macro-F1 and ROC-AUC in Table 18. This is common for small tabular datasets (about 3,100 training rows), where the relationships are largely monotonic and simpler models are already close to the achievable limit. Because all three models are evaluated on one 664-row test set, differences of one or two percentage points should not be over-interpreted. The MLP is kept as the required deep-learning model; the baselines are reported for context.

7.3 Result Visualizations and Interpretation

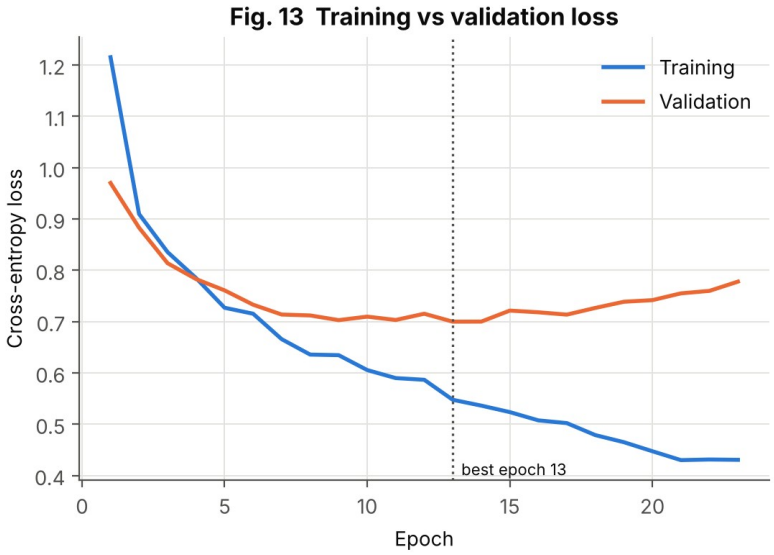


Figure 13: Training vs validation loss

Aspect	Explanation
Purpose	Show how learning progressed and detect over-fitting.
Variables	Epoch, cross-entropy loss (train and validation)
Chart type	Line chart with best-epoch marker
Why selected	Loss curves are the standard diagnostic of neural-network training.
Insight	Training loss keeps falling, but validation loss stops improving around epoch 13 and then rises, a classic sign of over-fitting. Early stopping ended training at epoch 23 and restored the weights from epoch 13.

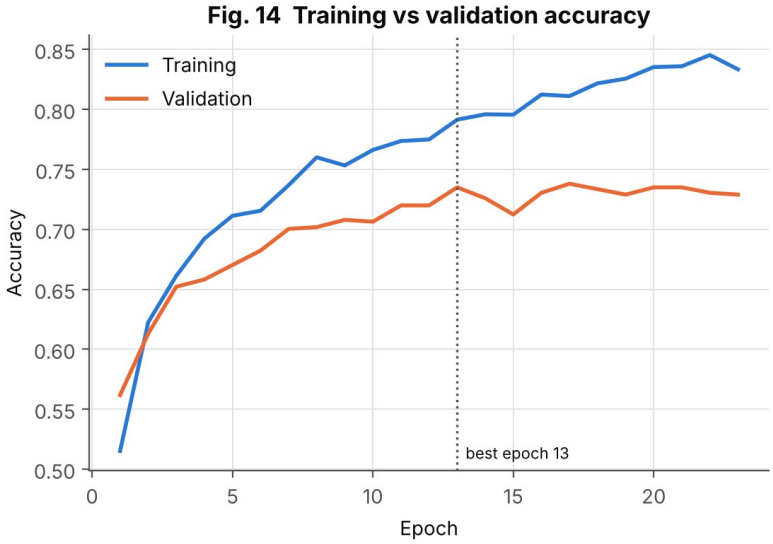


Figure 14: Training vs validation accuracy

Aspect	Explanation
Purpose	Show accuracy progress for both subsets.
Variables	Epoch, accuracy (train and validation)
Chart type	Line chart
Why selected	Accuracy is easier to interpret than loss and is requested by the assignment.
Insight	Training accuracy continues to climb while validation accuracy flattens, giving a widening gap after the early epochs – consistent with Figure 13. The gap is the reason dropout, L2 and early stopping were used.

Fig. 15 Confusion matrix on the test set

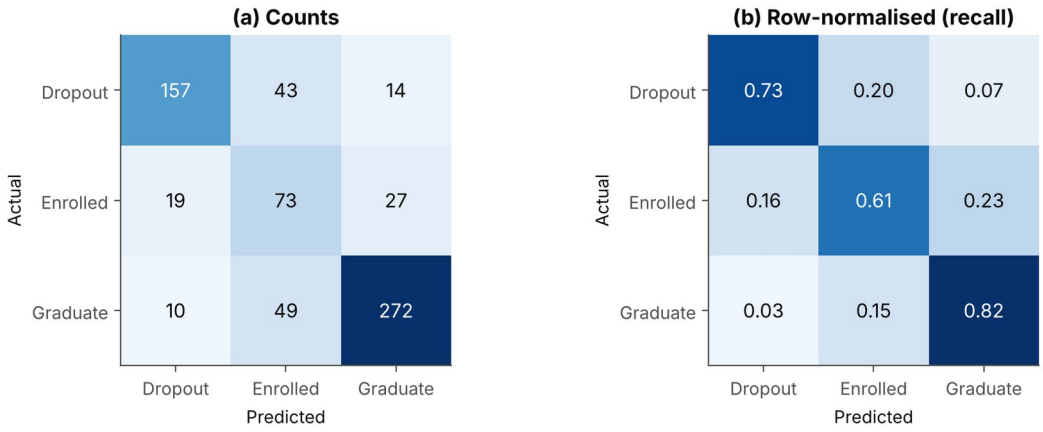


Figure 15: Confusion matrix on the test set

Aspect	Explanation
Purpose	Show which classes are confused with each other.
Variables	Actual vs predicted class (test set)
Chart type	Heat-map of counts and row-normalised recall
Why selected	A confusion matrix reveals per-class errors that a single accuracy figure hides.
Insight	Recall is 0.73 for Dropout, 0.61 for Enrolled and 0.82 for Graduate. Most mistakes involve the Enrolled class: 49 Graduates and 43 Dropouts were predicted as Enrolled, and 27 Enrolled students were predicted as Graduate. Enrolled has the lowest precision (0.44) and F1 (0.51); only 10 of 331 Graduates were predicted as Dropout, so the extreme confusions are rare.

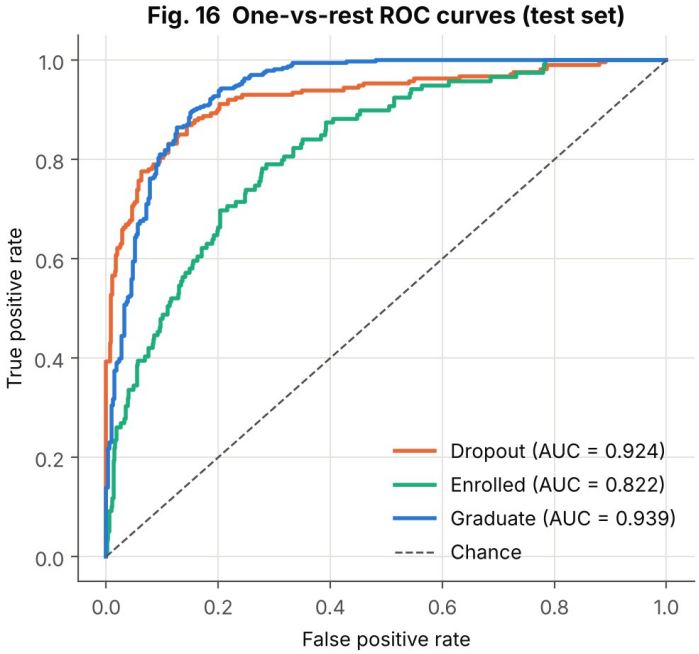


Figure 16: One-vs-rest ROC curves

Aspect	Explanation
Purpose	Show ranking quality of the predicted probabilities for every class at all thresholds.
Variables	False- and true-positive rates for each class
Chart type	ROC curves with AUC in the legend
Why selected	ROC/AUC is threshold-independent and is requested by the assignment; one-vs-rest extends it to three classes.
Insight	AUC is 0.939 for Graduate, 0.924 for Dropout and 0.822 for Enrolled (macro 0.895). All curves are well above the chance diagonal, but Enrolled is clearly the hardest class, consistent with the confusion matrix.

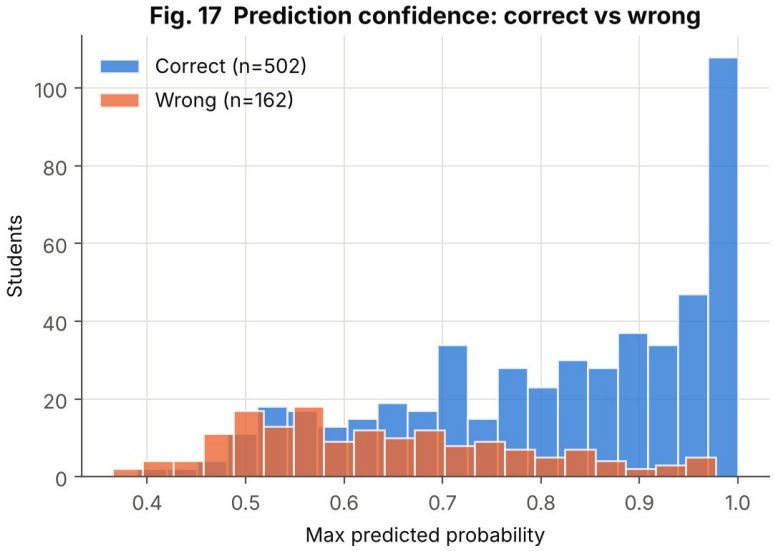


Figure 17: Prediction confidence: correct vs wrong

Aspect	Explanation
Purpose	Show whether the model is less confident when it is wrong (acts as an error-distribution view for a classifier).
Variables	Maximum predicted probability, correct vs wrong predictions
Chart type	Overlaid histograms
Why selected	If wrong predictions are low-confidence, uncertain cases can be flagged for human review.
Insight	Mean confidence is 0.82 for correct and 0.64 for wrong predictions. Wrong predictions are spread over lower confidence values, so low-confidence cases could be sent to a tutor, but the model is still sometimes confidently wrong.

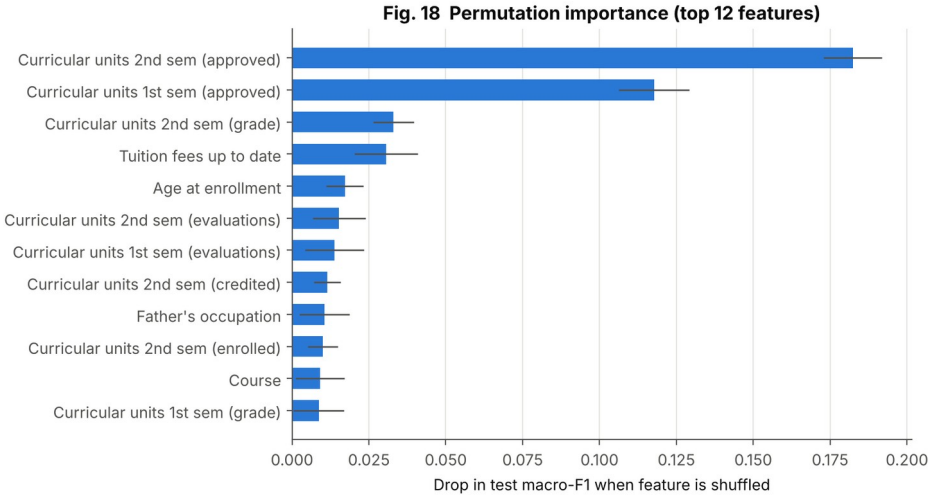


Figure 18: Permutation importance (top 12 features)

Aspect	Explanation
Purpose	Show which raw features the trained model depends on.
Variables	Raw features; drop in test macro-F1 after shuffling
Chart type	Horizontal bar chart with error bars (SD over 10 shuffles)
Why selected	Permutation importance is model-agnostic and works with one-hot encoded inputs when applied to raw columns.
Insight	Units approved in semester 2 (drop 0.182) and semester 1 (0.118) dominate; semester-2 grade, tuition-fee status and age follow. Admission grade and macro-economic indicators are not among the top features.

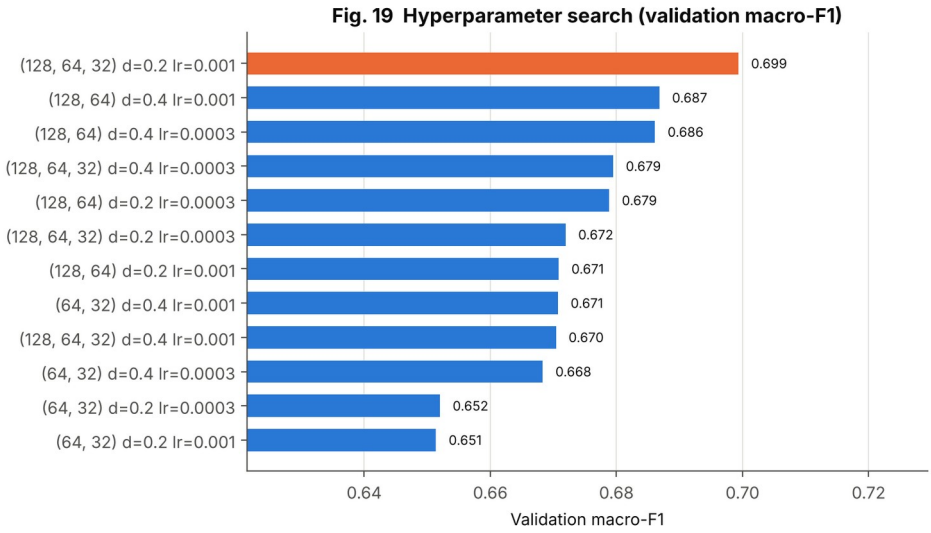


Figure 19: Hyperparameter search results

Aspect	Explanation
Purpose	Compare all 12 tested configurations.
Variables	Architecture, dropout, learning rate vs validation macro-F1
Chart type	Sorted horizontal bars; best configuration highlighted
Why selected	A ranked chart shows both the winner and how close the others are.
Insight	Validation macro-F1 ranges from 0.651 to 0.699. No single setting is dramatically better; the best (orange) is [128, 64, 32], dropout 0.2, learning rate 0.001.

8. Conclusion, Challenges and Future Scope

8.1 What Was Learned from the Data

- The dataset is clean in the sense of having no missing values or duplicates, but it contains meaningful zeros, skewed variables, many genuine outliers and strongly correlated features.
- Performance in the first two semesters is the clearest signal: 79% of students with no approved unit in semester 1 and 84% with none in semester 2 finally dropped out.
- Financial indicators are strongly associated with outcome: dropout was 86.55% when fees were not up to date, and scholarship holders dropped out less often (12.19% vs 38.71%).
- Admission grade and national macro-economic indicators carry much less information than later academic behaviour.

8.2 Model Performance

The tuned MLP reached 75.6% accuracy, macro-F1 0.715 and macro ROC-AUC 0.895 on the held-out test set. Dropout and Graduate are predicted well (F1 0.79 and 0.84); Enrolled is difficult (F1 0.51) because those students are, by definition, between the other two groups. The MLP did not outperform the random forest (80.3% accuracy) or logistic regression (76.7%).

8.3 Challenges and Limitations

- **Class imbalance and the 'Enrolled' class:** handled with class weights and macro metrics, but recall for Enrolled is still modest.
- **Over-fitting:** the network over-fits after about 13 epochs on 3,096 training rows; mitigated with dropout, L2, BatchNorm and early stopping.
- **Coded categories:** meanings of code values are not interpreted in this report; the UCI documentation must be consulted for domain explanation.
- **Single split and seed:** results come from one stratified split and one random seed; no repeated cross-validation was done, so small differences between models are not statistically established.
- **Timing of the strongest features:** semester-1 and semester-2 results are known only after those semesters, so the model is an early-warning tool for the second year, not at enrolment.
- **Single institution:** the data come from one Portuguese polytechnic, so results may not transfer to other institutions.
- **Data source for this study:** the CSV was obtained from public copies of the UCI dataset (identical across two copies) because the UCI site was not reachable from the analysis environment.
- **Ethics:** features such as gender and nationality should be used with care; a deployed system must be checked for fairness and used to offer support, not to penalise students.

8.4 Future Scope

- Build a **first-semester-only** model to give earlier warnings, and a model that uses only enrolment-time information.
- Use repeated stratified k-fold cross-validation with confidence intervals to compare models properly.
- Try gradient-boosted trees, embeddings for high-cardinality codes (Course, occupation) and focal loss or SMOTE for the Enrolled class.
- Explain predictions for individual students with SHAP, and calibrate predicted probabilities.
- Fit zero-inflated count models for the approved-units variables.
- Wrap the model in a simple dashboard (Plotly/Streamlit) for tutors, and monitor fairness across gender and nationality groups.

9. Literature Review and References

9.1 Short Literature Review

Dataset and application. Realinho et al. (2022) describe the dataset used here as a combination of demographic, socio-economic, macro-economic and academic records from the Polytechnic Institute of Portalegre, gathered at enrolment and at the end of the first and second semesters, and used to train models that estimate each student's risk of dropout or failure for a tutoring team. This project reuses the data openly released under a CC BY 4.0 licence (Realinho et al., 2021).

EDA and visualisation. Tukey (1977) established exploratory analysis – looking at data before modelling. Fry (2007) formalised the seven-stage visualisation process (acquire, parse, filter, mine, represent, refine, interact) that this report follows. Plotting and statistics were done with Matplotlib (Hunter, 2007), seaborn (Waskom, 2021) and SciPy (Virtanen et al., 2020).

Modelling. Goodfellow, Bengio and Courville (2016) describe multilayer perceptrons, dropout and regularisation; Srivastava et al. (2014) introduced dropout and Kingma and Ba (2015) the Adam optimiser used here. The baselines use scikit-learn (Pedregosa et al., 2011) and the random-forest method of Breiman (2001). The network was built with TensorFlow/Keras (Abadi et al., 2016).

9.2 References (APA 7th edition)

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., et al. (2016). TensorFlow: A system for large-scale machine learning. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, 265–283.

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32.

Fry, B. (2007). *Visualizing data: Exploring and explaining data with the Processing environment*. O'Reilly Media.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations (ICLR 2015)*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). Predicting student dropout and academic success. *Data*, *7*(11), 146. https://doi.org/10.3390/data7110146

Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2021). *Predict students' dropout and academic success* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(56), 1929–1958.

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.

Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272.

Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021.

10. Rubric Alignment

The table maps each assessment criterion in the assignment sheet to the evidence in this report and repository. It lists where the evidence is; it does not claim any marks.

Table 19: Rubric alignment

CO / Criterion (marks)	Report section	Evidence provided
CO3 – Selection and creation of visualizations (3)	4.1, 4.4, Figs. 1–12; Ch. 7, Figs. 13–19	19 figures of 10+ types (bar, histogram, boxplot, scatter, heat-map, pairplot, Q–Q, PCA, ROC, confusion matrix, line) chosen by data type; 2 interactive pages
CO3 – Quality and presentation (2)	4.2	Numbered titles, labelled axes, legends, direct labels, colour-blind-safe palette, consistent class colours
CO3 – Interpretation of visualizations (3)	4.4, 7.3	Purpose / variables / chart type / why / insight table for every figure
CO4 – Univariate analysis and distribution exploration (4)	2.5, 5.1, Figs. 1–5	Mean, median, mode, SD, quantiles, skew, kurtosis, histograms, boxplots, outlier analysis
CO4 – Discrete / continuous distributions (4)	5.2, Figs. 5–6	Normal fit + Q–Q + normality tests; log-normal comparison; Bernoulli/Binomial with simulation; Poisson fit and dispersion analysis
CO4 – Bivariate / multivariate analysis (4)	5.3, 5.4, Figs. 7–11	Kruskal–Wallis, chi-square with effect sizes, Spearman heat-map, VIF, PCA, pairplot, scatter
CO4 – Interpretation of patterns and findings (4)	5.5, 3.6	Plain-language findings table linked to evidence
CO5 – Real-world problem / application (3)	1.2–1.5, 2.2	Student dropout early-warning problem with a real dataset
CO5 – Application of EDA techniques (3)	Chapter 3	Missing/duplicate checks, outliers, encoding, scaling, split, leakage control
CO5 – Insights and interpretation (3)	3.6, 5.5, 4.4	Zero-grade finding, fee and scholarship effects, weak admission grade, over-dispersion
CO5 – Conclusion and recommendations (3)	Chapter 8	Findings, model performance, limitations, future scope
CO5 – GitHub repository and documentation (4)	Appendix B	Repository structure, README, requirements, dataset file, source code, report; public link to be added after upload
Enrichment – ML/DL (optional)	Ch. 6, 7	Tuned MLP, baselines, feature importance, ROC/AUC, confusion matrix, learning curves

Appendix A – Complete Python Source Code

Files are shown in the order in which the pipeline runs. Run everything with: python src/run_all.py

A.1 config.py – paths, seed, column groups, plot style

```
"""Project-wide configuration: paths, random seed, column groups and plot style."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")                      # headless backend (no display needed)
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = ROOT / "data" / "data.csv"
EDA_DIR = ROOT / "outputs" / "eda"
EVAL_DIR = ROOT / "outputs" / "model_evaluation"
MODEL_DIR = ROOT / "models"
for d in (EDA_DIR, EVAL_DIR, MODEL_DIR):
    d.mkdir(parents=True, exist_ok=True)

SEED = 42
TARGET = "Target"
CLASS_NAMES = ["Dropout", "Enrolled", "Graduate"]      # alphabetical = LabelEncoder order

# ---- Column groups (36 input features) -----------------------------------
BINARY_COLS = ["Daytime/evening attendance", "Displaced", "Educational special needs",
               "Debtor", "Tuition fees up to date", "Gender", "Scholarship holder",
               "International"]
NOMINAL_COLS = ["Marital status", "Application mode", "Course", "Previous qualification",
                "Nacionality", "Mother's qualification", "Father's qualification",
                "Mother's occupation", "Father's occupation"]      # integer codes, no order
# everything else is numeric (continuous grades, counts, macro-economic indicators)

# ---- Plot style: validated categorical slots 1-3 (dataviz skill palette) ----
CLASS_COLORS = {"Graduate": "#2a78d6", "Dropout": "#eb6834", "Enrolled": "#1baf7a"}
INK, INK2, GRID = "#0b0b0b", "#52514e", "#e4e3df"

def set_style():
    plt.rcParams.update({
        "figure.dpi": 110, "savefig.dpi": 200, "savefig.bbox": "tight",
        "font.size": 10, "axes.titlesize": 12, "axes.titleweight": "bold",
        "axes.labelsize": 10, "axes.edgecolor": INK2, "axes.labelcolor": INK,
        "text.color": INK, "xtick.color": INK2, "ytick.color": INK2,
        "axes.spines.top": False, "axes.spines.right": False,
        "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.7,
        "axes.axisbelow": True, "legend.frameon": False,
        "figure.facecolor": "white", "axes.facecolor": "white",
    })

def save_fig(fig, path):
    fig.savefig(path)
    plt.close(fig)
```

A.2 data_preprocessing.py – loading, cleaning, split, pipeline

```
"""Data loading, cleaning checks, splitting and the leakage-safe preprocessing pipeline."""
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from config import (DATA_PATH, TARGET, SEED, BINARY_COLS, NOMINAL_COLS, CLASS_NAMES)


def load_data(path=DATA_PATH) -> pd.DataFrame:
    """Load the UCI 'Predict Students' Dropout and Academic Success' CSV (';' separated)."""
    df = pd.read_csv(path, sep=";")
    df.columns = df.columns.str.strip()          # header contains stray whitespace/tab
    return df


def numeric_cols(df: pd.DataFrame):
    """Numeric features = all features that are neither binary flags nor nominal codes."""
    return [c for c in df.columns if c not in BINARY_COLS + NOMINAL_COLS + [TARGET]]


def clean_data(df: pd.DataFrame):
    """Remove duplicates; report missing values. Returns (clean_df, report dict)."""
    report = {"rows_before": len(df),
              "missing_cells": int(df.isna().sum().sum()),
              "duplicate_rows": int(df.duplicated().sum())}
    df = df.drop_duplicates().reset_index(drop=True)
    report["rows_after"] = len(df)
    return df, report


class Winsorizer(BaseEstimator, TransformerMixin):
    """Clip each column to its [lower, upper] percentile, learned on TRAINING data only."""
    def __init__(self, lower=1, upper=99):
```

```
        self.lower, self.upper = lower, upper

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.lo_ = np.percentile(X, self.lower, axis=0)
        self.hi_ = np.percentile(X, self.upper, axis=0)
        return self

    def transform(self, X):
        return np.clip(np.asarray(X, dtype=float), self.lo_, self.hi_)

    def get_feature_names_out(self, names=None):
        return np.asarray(names)


def build_preprocessor(df: pd.DataFrame) -> ColumnTransformer:
    """numeric -> winsorise -> standardise | nominal -> one-hot (rare levels grouped) | binary -> as is."""
    num = numeric_cols(df)
    return ColumnTransformer([
        ("num", Pipeline([("clip", Winsorizer(1, 99)), ("scale", StandardScaler())]), num),
        ("cat", OneHotEncoder(min_frequency=0.02, handle_unknown="infrequent_if_exist",
                              sparse_output=False), NOMINAL_COLS),
        ("bin", "passthrough", BINARY_COLS),
    ])


def split_data(df: pd.DataFrame, val_size=0.15, test_size=0.15):
    """Stratified 70/15/15 split of raw features; target encoded with a fixed class order."""
    le = LabelEncoder().fit(CLASS_NAMES)
    X, y = df.drop(columns=[TARGET]), pd.Series(le.transform(df[TARGET]), index=df.index)
    X_tr, X_tmp, y_tr, y_tmp = train_test_split(
        X, y, test_size=val_size + test_size, stratify=y, random_state=SEED)
    X_val, X_te, y_val, y_te = train_test_split(
        X_tmp, y_tmp, test_size=test_size / (val_size + test_size), stratify=y_tmp, random_state=SEED)
    return X_tr, X_val, X_te, y_tr, y_val, y_te
```

A.3 eda.py – statistics and distribution fitting

```
"""Statistical EDA: data inspection, outliers, univariate / bivariate / multivariate statistics,
distribution fitting. Writes CSV tables and a results.json used by the report."""
import json
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from config import EDA_DIR, TARGET, BINARY_COLS, NOMINAL_COLS, SEED
from data_preprocessing import load_data, clean_data, numeric_cols


def inspect(df):
    """Shape, dtypes, missing, duplicates, descriptive stats (mean/median/mode/std/quantiles)."""
    num = numeric_cols(df)
    dt = pd.DataFrame({"dtype": df.dtypes.astype(str), "n_unique": df.nunique(),
                       "missing": df.isna().sum()})
    dt.to_csv(EDA_DIR / "table_dtypes.csv")
    rows = []
    for c in num:
        s = df[c]
        rows.append({"feature": c, "mean": s.mean(), "median": s.median(),
                     "mode": s.mode().iloc[0], "std": s.std(), "min": s.min(),
                     "q05": s.quantile(.05), "q25": s.quantile(.25), "q75": s.quantile(.75),
                     "q95": s.quantile(.95), "max": s.max(), "skew": s.skew(),
                     "kurtosis": s.kurt()})
    desc = pd.DataFrame(rows).set_index("feature")
    desc.round(3).to_csv(EDA_DIR / "table_descriptive_numeric.csv")
    return dt, desc


def outlier_table(df):
    """IQR (1.5x) and |z|>3 outlier counts for numeric features with more than 10 distinct values."""
    rows = []
    for c in numeric_cols(df):
        s = df[c]
        if s.nunique() <= 10:
            continue
        q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1
        lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
        n_iqr = int(((s < lo) | (s > hi)).sum())
        n_z = int((np.abs((s - s.mean()) / s.std()) > 3).sum())
        rows.append({"feature": c, "lower_fence": lo, "upper_fence": hi,
                     "iqr_outliers": n_iqr, "iqr_pct": 100 * n_iqr / len(s), "z_gt_3": n_z})
    t = pd.DataFrame(rows).set_index("feature").sort_values("iqr_pct", ascending=False)
    t.round(3).to_csv(EDA_DIR / "table_outliers.csv")
    return t


def bivariate_tests(df):
    """Association of every feature with the 3-class target.
    numeric -> Kruskal-Wallis H (+ epsilon^2 effect size); binary/nominal -> chi-square (+ Cramer's V)."""
    n, k = len(df), df[TARGET].nunique()
    rows = []
    for c in numeric_cols(df):
        groups = [g[c].values for _, g in df.groupby(TARGET)]
        H, p = stats.kruskal(*groups)
```

```
        rows.append({"feature": c, "type": "numeric", "test": "Kruskal-Wallis",
                     "statistic": H, "p_value": p, "effect_size": (H - k + 1) / (n - k)})
    for c in BINARY_COLS + NOMINAL_COLS:
        ct = pd.crosstab(df[c], df[TARGET])
        chi2, p, dof, _ = stats.chi2_contingency(ct)
        v = np.sqrt(chi2 / (n * (min(ct.shape) - 1)))
        rows.append({"feature": c, "type": "binary" if c in BINARY_COLS else "nominal",
                     "test": "Chi-square", "statistic": chi2, "p_value": p, "effect_size": v})
    t = pd.DataFrame(rows).set_index("feature").sort_values("effect_size", ascending=False)
    t.to_csv(EDA_DIR / "table_bivariate_tests.csv")
    return t


def multivariate(df):
    """Spearman correlation matrix, VIF (multicollinearity) and PCA on numeric features."""
    num = numeric_cols(df)
    corr = df[num].corr(method="spearman")
    corr.round(3).to_csv(EDA_DIR / "table_spearman_corr.csv")
    X = StandardScaler().fit_transform(df[num])
    vif = {}
    for i, c in enumerate(num):
        r2 = LinearRegression().fit(np.delete(X, i, 1), X[:, i]).score(np.delete(X, i, 1), X[:, i])
        vif[c] = 1 / (1 - r2) if r2 < 1 else np.inf
    vif = pd.Series(vif, name="VIF").sort_values(ascending=False)
    vif.round(2).to_csv(EDA_DIR / "table_vif.csv")
    pca = PCA().fit(X)
    return corr, vif, pca.explained_variance_ratio_


def distribution_fits(df):
    """Continuous (normal / log-normal) and discrete (Bernoulli-Binomial / Poisson) fits."""
    out = {}
    # --- continuous: normality of admission grade; log-normal vs normal for age
    for c in ["Admission grade", "Previous qualification (grade)"]:
        s = df[c]; mu, sd = stats.norm.fit(s)
        k2, p_norm = stats.normaltest(s)
        out[c] = {"mean": mu, "std": sd, "skew": s.skew(), "excess_kurtosis": s.kurt(),
                  "dagostino_p": p_norm, "ks_stat": stats.kstest(s, stats.norm(mu, sd).cdf).statistic}
    age = df["Age at enrollment"]
    shape, loc, scale = stats.lognorm.fit(age, floc=0)
    mu, sd = stats.norm.fit(age)
    out["Age at enrollment"] = {
        "skew": age.skew(),
        "ks_normal": stats.kstest(age, stats.norm(mu, sd).cdf).statistic,
        "ks_lognormal": stats.kstest(age, stats.lognorm(shape, loc, scale).cdf).statistic}
    # --- discrete: Bernoulli/Binomial for dropout; Poisson for count variables
    p_drop = (df[TARGET] == "Dropout").mean()
    out["dropout_proportion"] = p_drop
    lo, hi = stats.binomtest(int((df[TARGET] == "Dropout").sum()), len(df)).proportion_ci(method="wilson")
    out["dropout_wilson_ci"] = [lo, hi]
    for c in ["Curricular units 1st sem (approved)", "Curricular units 2nd sem (approved)",
              "Curricular units 1st sem (without evaluations)"]:
        s = df[c]
        out[c] = {"mean": s.mean(), "variance": s.var(), "dispersion_index": s.var() / s.mean(),
                  "zero_share": float((s == 0).mean())}
    return out


def run():
    raw = load_data()
    df, clean_report = clean_data(raw)
    dt, desc = inspect(df)
    out = {"shape": list(raw.shape), "clean_report": clean_report,
           "class_counts": df[TARGET].value_counts().to_dict(),
           "class_pct": (100 * df[TARGET].value_counts(normalize=True)).round(2).to_dict(),
           "n_numeric": len(numeric_cols(df)), "n_binary": len(BINARY_COLS),
           "n_nominal": len(NOMINAL_COLS), "dtypes": dt["dtype"].value_counts().to_dict()}
    ot = outlier_table(df); out["outliers_top"] = ot.head(6).round(2).reset_index().to_dict("records")
    bt = bivariate_tests(df); out["bivariate_top"] = bt.head(12).reset_index().round(4).to_dict("records")
    corr, vif, evr = multivariate(df)
    out["vif_top"] = vif.head(6).round(2).to_dict()
    out["pca_var_ratio_first5"] = [round(float(x), 4) for x in evr[:5]]
    out["distribution_fits"] = distribution_fits(df)
    # dropout rates for key binary features (descriptive insight)
    rates = {}
    for c in ["Tuition fees up to date", "Debtor", "Scholarship holder", "Gender", "Displaced"]:
        rates[c] = (df.groupby(c)[TARGET].apply(lambda s: round(100 * (s == "Dropout").mean(), 2))).to_dict()
    out["dropout_rate_by_binary"] = {k: {str(a): b for a, b in v.items()} for k, v in rates.items()}
    # strongest numeric correlations with each other
    c = corr.where(np.triu(np.ones(corr.shape, bool), 1)).stack().sort_values(key=abs, ascending=False)
    out["top_corr_pairs"] = [[a, b, round(float(v), 3)] for (a, b), v in c.head(6).items()]
    for f in ["Curricular units 1st sem (grade)", "Curricular units 2nd sem (grade)"]:
        out.setdefault("grade_means_by_target", {})[f] = df.groupby(TARGET)[f].mean().round(2).to_dict()
    for f in ["Curricular units 2nd sem (approved)", "Age at enrollment", "Admission grade"]:
        out["grade_means_by_target"][f] = df.groupby(TARGET)[f].mean().round(2).to_dict()
    with open(EDA_DIR / "results_eda.json", "w") as fh:
        json.dump(out, fh, indent=2, default=lambda o: o.item() if hasattr(o, "item") else str(o))
    print(json.dumps(out, indent=1, default=str)[:3500])
    return df


if __name__ == "__main__":
    run()
```

A.4 visualizations.py – Figures 1–12 and interactive pages

```
"""EDA visualisations (Figures 1-12) + two interactive Plotly pages (stage 7: Interact)."""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from config import (EDA_DIR, TARGET, CLASS_COLORS, set_style, save_fig, SEED,
                    BINARY_COLS, CLASS_NAMES)
from data_preprocessing import load_data, clean_data, numeric_cols

ORDER = ["Dropout", "Enrolled", "Graduate"]
PAL = {k: CLASS_COLORS[k] for k in ORDER}


def fig01_target(df):
    """Class balance of the target (bar chart with counts and %)."""
    fig, ax = plt.subplots(figsize=(5.5, 3.6))
    cnt = df[TARGET].value_counts().reindex(ORDER)
    bars = ax.bar(cnt.index, cnt.values, color=[PAL[k] for k in cnt.index], width=0.55)
    for b, v in zip(bars, cnt.values):
        ax.text(b.get_x() + b.get_width() / 2, v + 30, f"{v}\n({100*v/len(df):.1f}%)", ha="center", fontsize=9)
    ax.set(title="Fig. 1  Student outcome (target) distribution", ylabel="Number of students",
           xlabel="Outcome at end of normal course duration", ylim=(0, cnt.max() * 1.18))
    ax.grid(axis="x", visible=False)
    save_fig(fig, EDA_DIR / "fig01_target_distribution.png")


def fig02_hist(df):
    """Histograms with mean (solid) and median (dashed) for six key numeric variables."""
    cols = ["Admission grade", "Previous qualification (grade)", "Age at enrollment",
            "Curricular units 1st sem (grade)", "Curricular units 2nd sem (grade)",
            "Curricular units 2nd sem (approved)"]
    fig, axes = plt.subplots(2, 3, figsize=(11, 6))
    for ax, c in zip(axes.ravel(), cols):
        s = df[c]
        ax.hist(s, bins=30, color="#2a78d6", edgecolor="white", linewidth=0.5)
        ax.axvline(s.mean(), color="#0b0b0b", lw=1.6, label=f"mean {s.mean():.1f}")
        ax.axvline(s.median(), color="#eb6834", lw=1.6, ls="--", label=f"median {s.median():.1f}")
        ax.set(title=c, xlabel=c, ylabel="Frequency"); ax.legend(fontsize=8)
        ax.title.set_fontsize(10)
    fig.suptitle("Fig. 2  Histograms of key numeric variables", fontweight="bold", y=1.0)
    fig.tight_layout(); save_fig(fig, EDA_DIR / "fig02_histograms.png")


def fig03_box(df):
    """Boxplots of key variables split by outcome (outliers shown as points)."""
    cols = ["Admission grade", "Age at enrollment", "Curricular units 1st sem (grade)",
            "Curricular units 2nd sem (grade)", "Curricular units 2nd sem (approved)"]
    fig, axes = plt.subplots(1, 5, figsize=(13, 3.8))
    for ax, c in zip(axes, cols):
        sns.boxplot(data=df, x=TARGET, y=c, order=ORDER, palette=PAL, hue=TARGET, legend=False,
                    width=0.6, linewidth=1, fliersize=2, ax=ax)
        ax.set_title(c, fontsize=9); ax.set_xlabel(""); ax.set_ylabel("")
        ax.tick_params(axis="x", labelsize=8)
    fig.suptitle("Fig. 3  Boxplots of key variables by student outcome", fontweight="bold", y=1.03)
    fig.tight_layout(); save_fig(fig, EDA_DIR / "fig03_boxplots_by_target.png")


def fig04_outliers(df):
    """Share of IQR outliers per feature (horizontal bars)."""
    t = pd.read_csv(EDA_DIR / "table_outliers.csv", index_col=0).sort_values("iqr_pct")
    fig, ax = plt.subplots(figsize=(7.5, 5.2))
    ax.barh(t.index, t["iqr_pct"], color="#2a78d6", height=0.65)
    for y, v in enumerate(t["iqr_pct"]):
        ax.text(v + 0.3, y, f"{v:.1f}%", va="center", fontsize=8)
    ax.set(title="Fig. 4  Share of IQR outliers per numeric feature",
           xlabel="Rows outside Q1 - 1.5 IQR / Q3 + 1.5 IQR (%)"); ax.grid(axis="y", visible=False)
    ax.set_xlim(0, t["iqr_pct"].max() * 1.15)
    save_fig(fig, EDA_DIR / "fig04_outlier_share.png")


def fig05_normal(df):
    """Continuous distribution: normal fit on admission grade + Q-Q plot."""
    s = df["Admission grade"]; mu, sd = stats.norm.fit(s)
    fig, (a, b) = plt.subplots(1, 2, figsize=(10, 3.9))
    a.hist(s, bins=40, density=True, color="#2a78d6", edgecolor="white", linewidth=0.4, label="Observed")
    x = np.linspace(s.min(), s.max(), 300)
    a.plot(x, stats.norm.pdf(x, mu, sd), color="#eb6834", lw=2, label=f"Normal fit (μ={mu:.1f}, σ={sd:.1f})")
    a.set(title="(a) Histogram vs fitted normal", xlabel="Admission grade", ylabel="Density"); a.legend()
    stats.probplot(s, dist="norm", plot=b)
    b.get_lines()[0].set(color="#2a78d6", markersize=3); b.get_lines()[1].set(color="#eb6834", lw=2)
    b.set(title="(b) Normal Q-Q plot", xlabel="Theoretical quantiles", ylabel="Ordered values")
    fig.suptitle("Fig. 5  Continuous distribution: Admission grade", fontweight="bold", y=1.03)
    fig.tight_layout(); save_fig(fig, EDA_DIR / "fig05_normal_fit_qq.png")


def fig06_discrete(df):
    """Discrete distributions: (a) Poisson fit to approved units, (b) Binomial(10, p) for dropouts."""
    s = df["Curricular units 1st sem (approved)"]; lam = s.mean()
    fig, (a, b) = plt.subplots(1, 2, figsize=(10.5, 3.9))
    k = np.arange(0, s.max() + 1)
    obs = s.value_counts(normalize=True).reindex(k, fill_value=0)
    a.bar(k - 0.2, obs.values, width=0.4, color="#2a78d6", label="Observed")
```

```
    a.bar(k + 0.2, stats.poisson.pmf(k, lam), width=0.4, color="#eb6834", label=f"Poisson (λ={lam:.2f})")
    a.set(title="(a) Units approved, 1st semester", xlabel="Units approved", ylabel="Probability"); a.legend()
    p = (df[TARGET] == "Dropout").mean(); n = 10
    rng = np.random.default_rng(SEED)
    sims = np.array([(df[TARGET].sample(n, random_state=int(r)) == "Dropout").sum()
                     for r in rng.integers(0, 10**6, 5000)])
    kk = np.arange(0, n + 1)
    emp = np.bincount(sims, minlength=n + 1) / len(sims)
    b.bar(kk - 0.2, emp, width=0.4, color="#2a78d6", label="Simulated (5000 samples of 10 students)")
    b.bar(kk + 0.2, stats.binom.pmf(kk, n, p), width=0.4, color="#eb6834", label=f"Binomial(10, p={p:.3f})")
    b.set(title="(b) Dropouts among 10 random students", xlabel="Number of dropouts", ylabel="Probability")
    b.legend(fontsize=8)
    fig.suptitle("Fig. 6  Discrete distributions: Poisson and Binomial", fontweight="bold", y=1.03)
    fig.tight_layout(); save_fig(fig, EDA_DIR / "fig06_discrete_distributions.png")


def fig07_corr(df):
    """Spearman correlation heatmap of numeric features (diverging palette, neutral midpoint)."""
    num = numeric_cols(df); corr = df[num].corr(method="spearman")
    short = [c.replace("Curricular units ", "CU ").replace("Previous qualification", "Prev. qual.")
             .replace(" sem", "") for c in num]
    fig, ax = plt.subplots(figsize=(9, 7.4))
    mask = np.triu(np.ones_like(corr, bool), 1)
    sns.heatmap(corr, mask=mask, cmap=sns.diverging_palette(40, 250, s=85, l=50, as_cmap=True),
                center=0, vmin=-1, vmax=1, annot=True, fmt=".2f", annot_kws={"size": 6},
                xticklabels=short, yticklabels=short, linewidths=0.4, linecolor="white",
                cbar_kws={"label": "Spearman ρ", "shrink": 0.7}, ax=ax)
    ax.grid(False); ax.set_title("Fig. 7  Spearman correlation heatmap (numeric features)")
    save_fig(fig, EDA_DIR / "fig07_correlation_heatmap.png")


def fig08_binary(df):
    """100% stacked bars: outcome mix across four binary risk/context factors."""
    cols = {"Tuition fees up to date": ("No (0)", "Yes (1)"), "Debtor": ("No (0)", "Yes (1)"),
            "Scholarship holder": ("No (0)", "Yes (1)"), "Gender": ("Code 0", "Code 1")}
    fig, axes = plt.subplots(1, 4, figsize=(12.5, 3.8), sharey=True)
    for ax, (c, labs) in zip(axes, cols.items()):
        ct = pd.crosstab(df[c], df[TARGET], normalize="index").reindex(columns=ORDER) * 100
        bottom = np.zeros(len(ct))
        for cls in ORDER:
            ax.bar(range(len(ct)), ct[cls], bottom=bottom, color=PAL[cls], width=0.6,
                   edgecolor="white", linewidth=1.2, label=cls)
            for i, v in enumerate(ct[cls]):
                if v > 7: ax.text(i, bottom[i] + v / 2, f"{v:.0f}%", ha="center", va="center", fontsize=8,
color="white")
            bottom += ct[cls].values
        ax.set_xticks(range(len(ct))); ax.set_xticklabels(labs); ax.set_title(c, fontsize=10)
        ax.grid(axis="x", visible=False)
    axes[0].set_ylabel("Share of students (%)")
    h, l = axes[0].get_legend_handles_labels()
    fig.legend(h, l, loc="lower center", ncol=3, bbox_to_anchor=(0.5, -0.06))
    fig.suptitle("Fig. 8  Outcome mix by binary factors", fontweight="bold", y=1.02)
    fig.tight_layout(); save_fig(fig, EDA_DIR / "fig08_outcome_by_binary_factors.png")


def fig09_scatter(df):
    """Scatterplot of 1st vs 2nd semester grade coloured by outcome (jittered, semi-transparent)."""
    fig, ax = plt.subplots(figsize=(6.4, 5.2))
    rng = np.random.default_rng(SEED)
    for cls in ["Graduate", "Enrolled", "Dropout"]:
        d = df[df[TARGET] == cls]
        ax.scatter(d["Curricular units 1st sem (grade)"] + rng.normal(0, .08, len(d)),
                   d["Curricular units 2nd sem (grade)"] + rng.normal(0, .08, len(d)),
                   s=12, alpha=0.45, color=PAL[cls], label=cls, edgecolor="none")
    ax.set(title="Fig. 9  First vs second semester grade", xlabel="1st semester grade (0-20)",
           ylabel="2nd semester grade (0-20)")
    ax.legend(markerscale=2.2, title="Outcome")
    save_fig(fig, EDA_DIR / "fig09_grade_scatter.png")


def fig10_pca(df):
    """PCA projection of standardised numeric features coloured by outcome."""
    num = numeric_cols(df); Z = StandardScaler().fit_transform(df[num])
    pca = PCA(2, random_state=SEED).fit(Z); P = pca.transform(Z)
    fig, ax = plt.subplots(figsize=(6.4, 5.2))
    for cls in ["Graduate", "Enrolled", "Dropout"]:
        m = (df[TARGET] == cls).values
        ax.scatter(P[m, 0], P[m, 1], s=10, alpha=0.4, color=PAL[cls], label=cls, edgecolor="none")
    ev = pca.explained_variance_ratio_ * 100
    ax.set(title="Fig. 10  PCA projection of numeric features", xlabel=f"PC1 ({ev[0]:.1f}% variance)",
           ylabel=f"PC2 ({ev[1]:.1f}% variance)")
    ax.legend(markerscale=2.5, title="Outcome")
    save_fig(fig, EDA_DIR / "fig10_pca.png")


def fig11_pair(df):
    """Pairplot of five key numeric variables coloured by outcome."""
    cols = ["Admission grade", "Age at enrollment", "Curricular units 1st sem (grade)",
            "Curricular units 2nd sem (grade)", "Curricular units 2nd sem (approved)"]
    short = {c: c.replace("Curricular units ", "CU ").replace(" sem", "") for c in cols}
    d = df[cols + [TARGET]].rename(columns=short)
    g = sns.pairplot(d, hue=TARGET, hue_order=ORDER, palette=PAL, corner=True,
                     plot_kws={"s": 8, "alpha": 0.35, "edgecolor": "none"}, diag_kind="kde", height=1.9)
    g.figure.suptitle("Fig. 11  Pairplot of key variables", fontweight="bold", y=1.01)
    g.figure.savefig(EDA_DIR / "fig11_pairplot.png"); plt.close(g.figure)


def fig12_course(df):
```

```
    """Dropout rate by course code (courses with >=100 students), with overall rate reference."""
    g = df.groupby("Course")[TARGET].agg(n="size", rate=lambda s: 100 * (s == "Dropout").mean())
    g = g[g.n >= 100].sort_values("rate")
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.barh(g.index.astype(str), g.rate, color="#eb6834", height=0.65)
    for y, (r, n) in enumerate(zip(g.rate, g.n)):
        ax.text(r + 0.6, y, f"{r:.0f}%  (n={n})", va="center", fontsize=8, bbox=dict(fc="white", ec="none", pad=0.8),
zorder=5)
    overall = 100 * (df[TARGET] == "Dropout").mean()
    ax.axvline(overall, color="#0b0b0b", ls="--", lw=1.2); ax.text(overall + .4, -0.9, f"overall {overall:.1f}%",
fontsize=8)
    ax.set(title="Fig. 12  Dropout rate by course code", xlabel="Dropout rate (%)", ylabel="Course code")
    ax.set_xlim(0, g.rate.max() * 1.25); ax.grid(axis="y", visible=False)
    save_fig(fig, EDA_DIR / "fig12_dropout_by_course.png")


def interactive(df):
    """Stage 7 (Interact): hover/zoom/filter pages built with Plotly."""
    import plotly.express as px
    d = df.copy(); d["Outcome"] = d[TARGET]
    f1 = px.scatter(d, x="Curricular units 1st sem (grade)", y="Curricular units 2nd sem (grade)",
                    color="Outcome", color_discrete_map=PAL, opacity=0.5,
                    hover_data=["Age at enrollment", "Admission grade", "Course", "Tuition fees up to date"],
                    title="Interactive: 1st vs 2nd semester grade (click legend to filter)")
    f1.write_html(EDA_DIR / "interactive_grades_scatter.html", include_plotlyjs="cdn")
    r = (d.groupby(["Course", "Outcome"]).size().reset_index(name="students"))
    f2 = px.bar(r, x="Course", y="students", color="Outcome", color_discrete_map=PAL,
                barmode="stack", title="Interactive: students by course code and outcome")
    f2.update_xaxes(type="category"); f2.write_html(EDA_DIR / "interactive_course_outcomes.html",
include_plotlyjs="cdn")


def run():
    set_style()
    df, _ = clean_data(load_data())
    for f in (fig01_target, fig02_hist, fig03_box, fig04_outliers, fig05_normal, fig06_discrete,
              fig07_corr, fig08_binary, fig09_scatter, fig10_pca, fig11_pair, fig12_course, interactive):
        f(df); print("done", f.__name__)


if __name__ == "__main__":
    run()
```

A.5 train_model.py – tuning, training, baselines

```
"""Train the MLP: leakage-safe preprocessing, grid-search hyperparameter tuning on the
VALIDATION set, baselines (logistic regression, random forest), save model + artefacts."""
import itertools, json, os, time
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import joblib
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.utils.class_weight import compute_class_weight
from config import MODEL_DIR, EVAL_DIR, SEED, CLASS_NAMES
from data_preprocessing import load_data, clean_data, split_data, build_preprocessor

BATCH_SIZE, MAX_EPOCHS, PATIENCE = 64, 100, 10
GRID = {"hidden": [(64, 32), (128, 64), (128, 64, 32)],
        "dropout": [0.2, 0.4],
        "lr": [1e-3, 3e-4]}


def build_mlp(n_in, hidden, dropout, lr, n_classes=3):
    """MLP: [Dense(ReLU) -> BatchNorm -> Dropout] x L -> Dense(softmax)."""
    layers = [tf.keras.layers.Input((n_in,))]
    for h in hidden:
        layers += [tf.keras.layers.Dense(h, activation="relu",
                                         kernel_regularizer=tf.keras.regularizers.l2(1e-4)),
                   tf.keras.layers.BatchNormalization(),
                   tf.keras.layers.Dropout(dropout)]
    layers.append(tf.keras.layers.Dense(n_classes, activation="softmax"))
    m = tf.keras.Sequential(layers)
    m.compile(optimizer=tf.keras.optimizers.Adam(lr), loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
    return m


def fit_one(Xtr, ytr, Xval, yval, cw, hidden, dropout, lr):
    tf.keras.utils.set_random_seed(SEED)
    m = build_mlp(Xtr.shape[1], hidden, dropout, lr)
    es = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=PATIENCE,
                                          restore_best_weights=True)
    h = m.fit(Xtr, ytr, validation_data=(Xval, yval), epochs=MAX_EPOCHS, batch_size=BATCH_SIZE,
              class_weight=cw, callbacks=[es], verbose=0)
    pred = m.predict(Xval, verbose=0).argmax(1)          # uses restored best-val-loss weights
    return m, h, f1_score(yval, pred, average="macro"), float((pred == yval).mean()), min(h.history["val_loss"])


def run():
    df, _ = clean_data(load_data())
    X_tr, X_val, X_te, y_tr, y_val, y_te = split_data(df)
    pre = build_preprocessor(X_tr).fit(X_tr)             # fitted on TRAIN only (no leakage)
```

```
    Xtr, Xval, Xte = (pre.transform(a).astype("float32") for a in (X_tr, X_val, X_te))
    ytr, yval, yte = y_tr.values, y_val.values, y_te.values
    joblib.dump(pre, MODEL_DIR / "preprocessor.joblib")
    feat_names = list(pre.get_feature_names_out())
    sizes = {"train": len(ytr), "val": len(yval), "test": len(yte), "n_input_features": Xtr.shape[1],
             "n_raw_features": X_tr.shape[1]}
    print(sizes)
    cw = dict(enumerate(compute_class_weight("balanced", classes=np.arange(3), y=ytr)))

    # ---- grid search on validation set ----------------------------------------------------
    rows, best = [], None
    for hidden, dr, lr in itertools.product(GRID["hidden"], GRID["dropout"], GRID["lr"]):
        t0 = time.time()
        m, h, f1, vacc, vl = fit_one(Xtr, ytr, Xval, yval, cw, hidden, dr, lr)
        rec = {"hidden": str(hidden), "dropout": dr, "lr": lr, "epochs_run": len(h.history["loss"]),
               "val_macro_f1": f1, "val_loss": vl, "val_acc": vacc,
               "seconds": round(time.time() - t0, 1)}
        rows.append(rec); print(rec)
        if best is None or (f1, -vl) > (best["f1"], -best["vl"]):
            best = {"f1": f1, "vl": vl, "model": m, "hist": h.history,
                    "cfg": {"hidden": list(hidden), "dropout": dr, "lr": lr}}
    pd.DataFrame(rows).to_csv(EVAL_DIR / "tuning_results.csv", index=False)

    # ---- baselines ------------------------------------------------------------------------
    base = {}
    for name, mdl in {"Logistic regression": LogisticRegression(max_iter=2000, class_weight="balanced",
                                                                random_state=SEED),
                      "Random forest": RandomForestClassifier(300, class_weight="balanced",
                                                              random_state=SEED, n_jobs=-1)}.items():
        mdl.fit(Xtr, ytr); joblib.dump(mdl, MODEL_DIR / f"baseline_{name.split()[0].lower()}.joblib")
        base[name] = {"val_macro_f1": f1_score(yval, mdl.predict(Xval), average="macro")}

    best["model"].save(MODEL_DIR / "mlp_student_outcome.keras")
    json.dump({"best_config": best["cfg"], "batch_size": BATCH_SIZE, "max_epochs": MAX_EPOCHS,
               "early_stopping_patience": PATIENCE, "epochs_run": len(best["hist"]["loss"]),
               "val_macro_f1": best["f1"], "sizes": sizes, "baselines_val": base,
               "class_weights": {CLASS_NAMES[k]: float(v) for k, v in cw.items()},
               "n_params": int(best["model"].count_params())},
              open(EVAL_DIR / "training_summary.json", "w"), indent=2)
    json.dump({k: [float(x) for x in v] for k, v in best["hist"].items()},
              open(EVAL_DIR / "history.json", "w"))
    json.dump(feat_names, open(EVAL_DIR / "feature_names.json", "w"))
    print("best:", best["cfg"], "val macro-F1", round(best["f1"], 4))


if __name__ == "__main__":
    run()
```

A.6 evaluate_model.py – evaluation, Figures 13–19

```
"""Evaluate the saved MLP on the held-out TEST set: metrics, 5 result figures, permutation importance."""
import json, os
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support, f1_score,
                             confusion_matrix, roc_curve, auc, roc_auc_score, classification_report)
from sklearn.preprocessing import label_binarize
from config import MODEL_DIR, EVAL_DIR, SEED, CLASS_NAMES, CLASS_COLORS, set_style, save_fig
from data_preprocessing import load_data, clean_data, split_data

CC = [CLASS_COLORS[c] for c in CLASS_NAMES]


def metrics(y, prob):
    pred = prob.argmax(1)
    p, r, f, _ = precision_recall_fscore_support(y, pred, average="macro", zero_division=0)
    return {"accuracy": accuracy_score(y, pred), "macro_precision": p, "macro_recall": r,
            "macro_f1": f, "weighted_f1": f1_score(y, pred, average="weighted"),
            "roc_auc_ovr_macro": roc_auc_score(y, prob, multi_class="ovr", average="macro")}


def run():
    set_style()
    df, _ = clean_data(load_data())
    X_tr, X_val, X_te, y_tr, y_val, y_te = split_data(df)
    pre = joblib.load(MODEL_DIR / "preprocessor.joblib")
    mlp = tf.keras.models.load_model(MODEL_DIR / "mlp_student_outcome.keras")
    hist = json.load(open(EVAL_DIR / "history.json"))
    summ = json.load(open(EVAL_DIR / "training_summary.json"))
    Xte, Xval = (pre.transform(a).astype("float32") for a in (X_te, X_val))
    yte, yval = y_te.values, y_val.values
    prob = mlp.predict(Xte, verbose=0); pred = prob.argmax(1)
    res = {"mlp_test": metrics(yte, prob), "mlp_val": metrics(yval, mlp.predict(Xval, verbose=0))}
    res["per_class_report"] = classification_report(yte, pred, target_names=CLASS_NAMES, output_dict=True)
    cm = confusion_matrix(yte, pred); res["confusion_matrix"] = cm.tolist()
    # baselines on the same test set
    comp = {"MLP (proposed)": res["mlp_test"]}
    for name, f in [("Logistic regression", "baseline_logistic.joblib"), ("Random forest", "baseline_random.joblib")]:
        m = joblib.load(MODEL_DIR / f); comp[name] = metrics(yte, m.predict_proba(pre.transform(X_te)))
    res["comparison_test"] = comp
    pd.DataFrame(comp).T.round(4).to_csv(EVAL_DIR / "table_model_comparison_test.csv")
```

```
    # ---- Fig 13/14: learning curves ------------------------------------------------------
    ep = np.arange(1, len(hist["loss"]) + 1); best_ep = int(np.argmin(hist["val_loss"])) + 1
    res["best_epoch"] = best_ep; res["epochs_run"] = len(ep)
    for key, title, yl, name in [("loss", "Fig. 13  Training vs validation loss", "Cross-entropy loss",
"fig13_loss_vs_epoch.png"),
                                 ("accuracy", "Fig. 14  Training vs validation accuracy", "Accuracy",
"fig14_accuracy_vs_epoch.png")]:
        fig, ax = plt.subplots(figsize=(6.2, 4))
        ax.plot(ep, hist[key], color="#2a78d6", lw=2, label="Training")
        ax.plot(ep, hist["val_" + key], color="#eb6834", lw=2, label="Validation")
        ax.axvline(best_ep, color="#52514e", ls=":", lw=1.2); ax.text(best_ep + .3, ax.get_ylim()[0] + .01, f"best
epoch {best_ep}", fontsize=8)
        ax.set(title=title, xlabel="Epoch", ylabel=yl); ax.legend()
        save_fig(fig, EVAL_DIR / name)

    # ---- Fig 15: confusion matrix (counts + row-normalised) ----------------------------
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 4))
    for ax, data, fmt, ttl in [(axes[0], cm, "d", "(a) Counts"), (axes[1], cm / cm.sum(1, keepdims=True), ".2f", "(b)
Row-normalised (recall)")]:
        im = ax.imshow(data, cmap="Blues", vmin=0)
        for i in range(3):
            for j in range(3):
                v = data[i, j]; ax.text(j, i, format(v, fmt), ha="center", va="center",
                                        color="white" if v > data.max() * .55 else "#0b0b0b", fontsize=11)
        ax.set_xticks(range(3)); ax.set_yticks(range(3)); ax.set_xticklabels(CLASS_NAMES);
ax.set_yticklabels(CLASS_NAMES)
        ax.set(title=ttl, xlabel="Predicted", ylabel="Actual"); ax.grid(False)
    fig.suptitle("Fig. 15  Confusion matrix on the test set", fontweight="bold", y=1.03); fig.tight_layout()
    save_fig(fig, EVAL_DIR / "fig15_confusion_matrix.png")

    # ---- Fig 16: one-vs-rest ROC curves -------------------------------------------------
    Y = label_binarize(yte, classes=[0, 1, 2]); res["roc_auc_per_class"] = {}
    fig, ax = plt.subplots(figsize=(5.6, 5))
    for i, c in enumerate(CLASS_NAMES):
        fpr, tpr, _ = roc_curve(Y[:, i], prob[:, i]); a = auc(fpr, tpr); res["roc_auc_per_class"][c] = a
        ax.plot(fpr, tpr, color=CC[i], lw=2, label=f"{c} (AUC = {a:.3f})")
    ax.plot([0, 1], [0, 1], color="#52514e", ls="--", lw=1, label="Chance")
    ax.set(title="Fig. 16  One-vs-rest ROC curves (test set)", xlabel="False positive rate", ylabel="True positive
rate")
    ax.legend(loc="lower right"); save_fig(fig, EVAL_DIR / "fig16_roc_curves.png")

    # ---- Fig 17: confidence distribution of correct vs wrong predictions ----------------
    conf = prob.max(1); ok = pred == yte
    fig, ax = plt.subplots(figsize=(6.2, 4))
    ax.hist(conf[ok], bins=20, alpha=.8, color="#2a78d6", label=f"Correct (n={ok.sum()})", edgecolor="white")
    ax.hist(conf[~ok], bins=20, alpha=.8, color="#eb6834", label=f"Wrong (n={(~ok).sum()})", edgecolor="white")
    ax.set(title="Fig. 17  Prediction confidence: correct vs wrong", xlabel="Max predicted probability",
ylabel="Students")
    ax.legend(); save_fig(fig, EVAL_DIR / "fig17_confidence_distribution.png")
    res["mean_conf_correct"], res["mean_conf_wrong"] = float(conf[ok].mean()), float(conf[~ok].mean())

    # ---- Fig 18: permutation importance on RAW features (drop in macro-F1) -------------
    rng = np.random.default_rng(SEED); base = f1_score(yte, pred, average="macro"); imp = {}
    for col in X_te.columns:
        drops = []
        for _ in range(10):
            Xp = X_te.copy(); Xp[col] = rng.permutation(Xp[col].values)
            drops.append(base - f1_score(yte, mlp.predict(pre.transform(Xp).astype("float32"), verbose=0).argmax(1),
average="macro"))
        imp[col] = (np.mean(drops), np.std(drops))
    imp = pd.DataFrame(imp, index=["mean_drop", "std"]).T.sort_values("mean_drop", ascending=False)
    imp.round(4).to_csv(EVAL_DIR / "table_permutation_importance.csv")
    top = imp.head(12).iloc[::-1]
    fig, ax = plt.subplots(figsize=(7.2, 4.8))
    ax.barh(top.index, top.mean_drop, xerr=top["std"], color="#2a78d6", height=.65, error_kw={"lw": 1, "ecolor":
"#52514e"})
    ax.set(title="Fig. 18  Permutation importance (top 12 features)", xlabel="Drop in test macro-F1 when feature is
shuffled")
    ax.grid(axis="y", visible=False); save_fig(fig, EVAL_DIR / "fig18_permutation_importance.png")
    res["top_importance"] = imp.head(10).round(4).reset_index().rename(columns={"index": "feature"}).to_dict("records")

    # ---- Fig 19: tuning results ---------------------------------------------------------
    t = pd.read_csv(EVAL_DIR / "tuning_results.csv")
    t["cfg"] = t.hidden + " d=" + t.dropout.astype(str) + " lr=" + t.lr.astype(str)
    t = t.sort_values("val_macro_f1")
    fig, ax = plt.subplots(figsize=(7.4, 4.8))
    colors = ["#eb6834" if v == t.val_macro_f1.max() else "#2a78d6" for v in t.val_macro_f1]
    ax.barh(t.cfg, t.val_macro_f1, color=colors, height=.65)
    for y, v in enumerate(t.val_macro_f1): ax.text(v + .002, y, f"{v:.3f}", va="center", fontsize=8)
    ax.set(title="Fig. 19  Hyperparameter search (validation macro-F1)", xlabel="Validation macro-F1",
           xlim=(t.val_macro_f1.min() - .03, t.val_macro_f1.max() + .03)); ax.grid(axis="y", visible=False)
    save_fig(fig, EVAL_DIR / "fig19_tuning_results.png")

    json.dump(res, open(EVAL_DIR / "results_model.json", "w"), indent=2, default=float)
    print(json.dumps({k: res[k] for k in ["mlp_test", "mlp_val", "comparison_test", "roc_auc_per_class", "best_epoch",
"epochs_run"]}, indent=1, default=float))
    print(classification_report(yte, pred, target_names=CLASS_NAMES)); print(cm); print(imp.head(10))


if __name__ == "__main__":
    run()
```

A.7 diagrams.py – workflow and architecture diagrams

```
"""Draw two explanatory diagrams: (1) project workflow, (2) MLP architecture."""
import json
```

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
from config import EVAL_DIR, EDA_DIR, set_style, save_fig, INK, INK2


def box(ax, x, y, w, h, text, fc="#e8f0fb", ec="#2a78d6", fs=9, bold=False):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=0.08",
                                fc=fc, ec=ec, lw=1.3))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=fs,
            fontweight="bold" if bold else "normal", color=INK)


def arrow(ax, x1, y1, x2, y2):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1),
                arrowprops=dict(arrowstyle="-|>", color=INK2, lw=1.3))


def workflow():
    fig, ax = plt.subplots(figsize=(10, 3.9)); ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 4)
    top = [("1. Acquire\nUCI dataset\n4,424 × 37", 0.1), ("2. Parse &\nclean\ntypes, duplicates,\nmissing values",
2.1),
           ("3. EDA &\nstatistics\nunivariate →\nmultivariate", 4.1), ("4. Visualise\n12 figures +\n2 interactive",
6.1),
           ("5. Split\n70 / 15 / 15\nstratified", 8.1)]
    for t, x in top:
        box(ax, x, 2.3, 1.75, 1.35, t, fs=8.5)
    for i in range(4):
        arrow(ax, top[i][1] + 1.77, 2.97, top[i + 1][1] - 0.02, 2.97)
    bot = [("6. Preprocess (train-fit)\nwinsorise → scale,\none-hot, binary", 0.1), ("7. Tune MLP\n12 configurations
on\nvalidation set", 2.65),
           ("8. Train final MLP\nearly stopping,\nclass weights", 5.2), ("9. Evaluate on test\nmetrics, ROC, confusion\
nmatrix, importance", 7.75)]
    for t, x in bot:
        box(ax, x, 0.35, 2.1, 1.25, t, fc="#fdeee6", ec="#eb6834", fs=8.5)
    for i in range(3):
        arrow(ax, bot[i][1] + 2.12, 0.97, bot[i + 1][1] - 0.02, 0.97)
    arrow(ax, 8.97, 2.28, 1.15, 1.62)
    ax.text(5.0, 3.95, "Diagram 1  End-to-end project workflow", ha="center", fontsize=12, fontweight="bold")
    save_fig(fig, EDA_DIR / "diagram1_workflow.png")


def architecture():
    s = json.load(open(EVAL_DIR / "training_summary.json"))
    n_in = s["sizes"]["n_input_features"]; hid = s["best_config"]["hidden"]; dr = s["best_config"]["dropout"]
    fig, ax = plt.subplots(figsize=(10, 3.4)); ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 3.4)
    blocks = [(f"Input\n{n_in} features", "#e8f0fb", "#2a78d6")]
    for h in hid:
        blocks.append((f"Dense {h}\nReLU + L2\nBatchNorm\nDropout {dr}", "#eaf6f0", "#1baf7a"))
    blocks.append(("Dense 3\nSoftmax", "#fdeee6", "#eb6834"))
    blocks.append(("Output\nDropout /\nEnrolled /\nGraduate", "#e8f0fb", "#2a78d6"))
    n = len(blocks); w = 1.4; gap = (10 - 0.3 - n * w) / (n - 1)
    for i, (t, fc, ec) in enumerate(blocks):
        x = 0.15 + i * (w + gap); box(ax, x, 0.7, w, 1.6, t, fc=fc, ec=ec, fs=8.5)
        if i < n - 1:
            arrow(ax, x + w + 0.02, 1.5, x + w + gap - 0.02, 1.5)
    ax.text(5, 3.0, f"Diagram 2  MLP architecture ({s['n_params']:,} parameters)", ha="center", fontsize=12,
fontweight="bold")
    save_fig(fig, EVAL_DIR / "diagram2_mlp_architecture.png")


if __name__ == "__main__":
    set_style(); workflow(); architecture()
```

A.8 run_all.py – run the whole pipeline

```
"""Run the whole pipeline end-to-end:  python src/run_all.py"""
import sys, pathlib
sys.path.insert(0, str(pathlib.Path(__file__).parent))
import eda, visualizations, train_model, evaluate_model

if __name__ == "__main__":
    eda.run()                # tables + statistics
    visualizations.run()     # Figures 1-12 + interactive pages
    train_model.run()        # preprocessing, tuning, training, baselines
    evaluate_model.run()     # test metrics + Figures 13-19
```

Appendix B – GitHub Repository and Checklist

B.1 Repository Structure

```
student-performance-analysis/
├── data/
│   └── data.csv                    # UCI dataset 697 (semicolon-separated, 4,424 x 37)
├── src/
│   ├── config.py                   # paths, seed, column groups, plot style
│   ├── data_preprocessing.py       # load, clean, split, leakage-safe pipeline
│   ├── eda.py                      # tables + statistical tests + distribution fits
│   ├── visualizations.py           # Figures 1-12 + interactive HTML pages
│   ├── train_model.py              # grid search, MLP training, baselines
│   ├── evaluate_model.py           # test metrics + Figures 13-19
│   ├── diagrams.py                 # workflow / architecture diagrams
│   └── run_all.py                  # runs the full pipeline
├── outputs/
│   ├── eda/                        # EDA figures, tables, interactive pages
│   └── model_evaluation/           # evaluation figures, metrics JSON/CSV
├── models/                         # saved MLP (.keras) + preprocessor (.joblib)
├── report/                         # Word + PDF report, report builder script
├── requirements.txt
├── README.md
└── .gitignore
```

Table 20: Purpose of each folder and file

Path	Purpose
data/	The dataset used (or a link to it). CC BY 4.0 – keep the citation in the README
src/	All source code, one module per stage so it is easy to read and to explain in a viva
outputs/eda, outputs/model_evaluation	Generated figures, tables and metrics, so the report can be checked without re-running
models/	Trained MLP and fitted preprocessor. The 36 MB random-forest baseline is excluded by .gitignore and is re-created by run_all.py
report/	Final report (DOCX and PDF) and the script that builds it from the saved results
requirements.txt	Exact library versions used
README.md	Project summary, results, how to run
.gitignore	Keeps caches, secrets and large files out of the repository

B.2 README Outline

- Title, one-paragraph summary and student details (RITHI R, 24AD091, U21ADP05)
- Problem statement and objectives
- Dataset: name, UCI URL, DOI, licence, size, target
- Project structure (tree above)
- How to run: pip install -r requirements.txt, then python src/run_all.py
- Key results table (accuracy, macro-F1, ROC-AUC) and 3–4 sample figures
- Key insights (5 bullets)
- Limitations and future work
- References / citation of the dataset
- Link to the report in report/

B.3 Commands to Publish

```
cd student-performance-analysis
git init
git add .
git commit -m "Student dropout EDA and MLP case study"
git branch -M main
git remote add origin https://github.com/<your-github-id>/student-performance-analysis.git
```

```
git push -u origin main
```

B.4 GitHub Submission Checklist

Table 21: Checklist before submitting the link

✔	Item
☐	Repository is **public** and created under your own GitHub ID
☐	Project report (PDF and Word) is inside report/
☐	Source code (.py) is inside src/ and runs with python src/run_all.py
☐	Dataset file or dataset link is present (data/data.csv + UCI link in README)
☐	README.md has summary, dataset, how to run, results and limitations
☐	No passwords, tokens, .env files or personal data are committed
☐	No very large files (> 50 MB); models/baseline_random.joblib is ignored
☐	Public GitHub link pasted into the Google Form (https://forms.gle/kT9DU5aERo2peTNh8) or emailed to the coordinator

Appendix C – Final Checklist Against the Assignment Requirements

Table 22: Requirement checklist

Requirement from assignment sheet	Status	Where
Real-world dataset with ≥ 15 features	Done – 36 features	2.1–2.4
Load data and perform EDA	Done	Ch. 3, 5
Handle missing values, outliers, duplicates	Done – none missing, none duplicated, outliers analysed and winsorised	3.2, 3.3
Cleaning, transformation, encoding, scaling	Done	3.4
Train / validation / test split	Done – 70/15/15 stratified	3.5
Minimum 5 visualisations covering all 7 stages	Done – 12 EDA + 7 model figures; stages mapped in Table 8	4.1, 4.4
Mention what each visual shows, why created, insights	Done – explanation table per figure	4.4, 7.3
At least one deep-learning model	Done – MLP	Ch. 6
Feature analysis, importance, basic tuning	Done	6.5, 6.6
3–4 result visualisations (loss, accuracy, confusion matrix, ROC/AUC)	Done – Figs. 13–19	7.3
Conclusion, challenges, future scope	Done	Ch. 8
3–5 references, APA/IEEE	Done – 13 APA references	9.2
Report format (title page … appendix)	Done	Whole report
Well-commented code with loading, EDA, preprocessing, model, evaluation	Done	Appendix A
GitHub repository, README, dataset link, report, source code	Prepared locally; **upload and public link still to be done by the student**	Appendix B